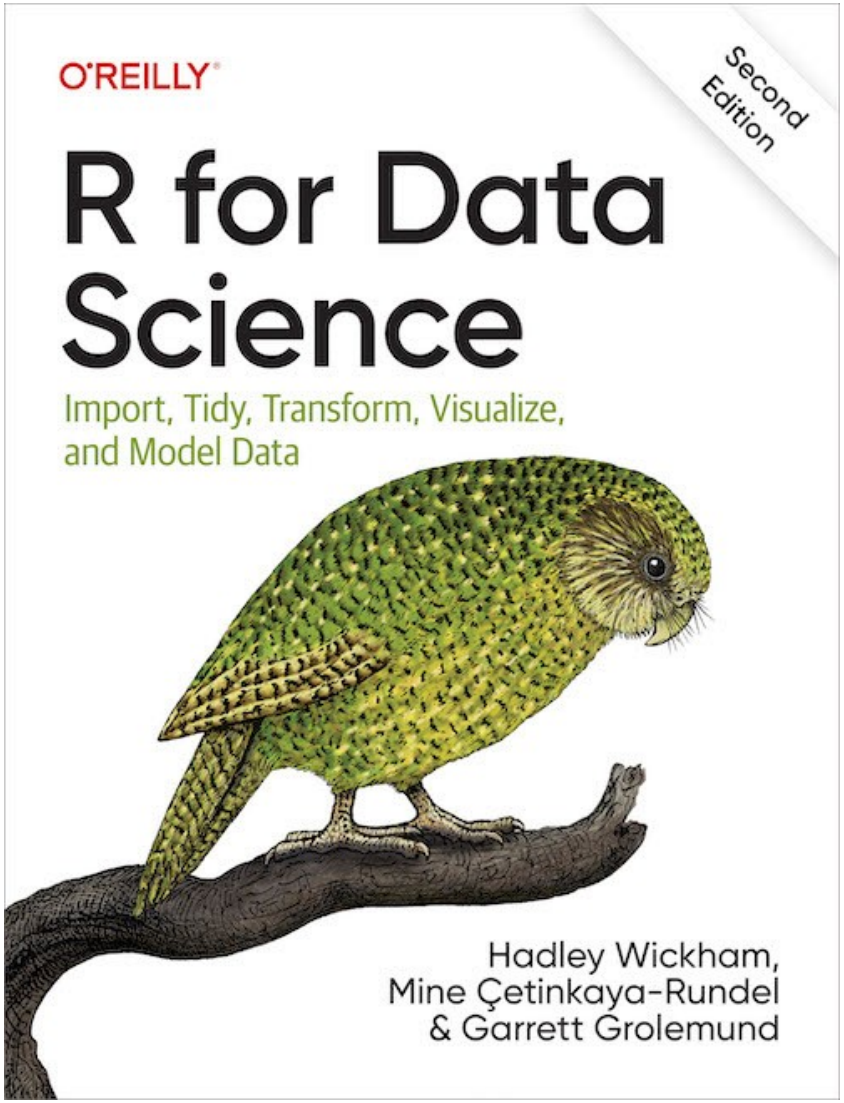
Data Science
CS301

Exploratory First Steps, Continued

Week 5
Fall 2025
Oliver BONHAM-CARTER

Where in the Web?



Web:

Chap 10: Exploratory Data Analysis

– https://r4ds.hadley.nz/eda

Missing Data Points?

Missing Data Entries

- Missing data in R appears as ***NA***.
- *NA* is not a string or a numeric value, but an indicator of missing data.
- Let's create vectors with missing values to test

```
library(tidyverse)
library(tibble)
x1 <- c(1, 4, 3, NA, 7)
x2 <- c("a", "B", NA, "NA")
is.na(x1)
is.na(x2)
```

Spot missing data

Missing Data Entries

- What to do when elements of your data go missing?
- **Why not just DROP the ENTIRE ROW, as well as to drop all the value contained by its other variables as well??**

```
diamonds2 <- diamonds  %>%  filter(between(y, 3, 20))
```

This is a shortcut for y >= 3 & y <= 20

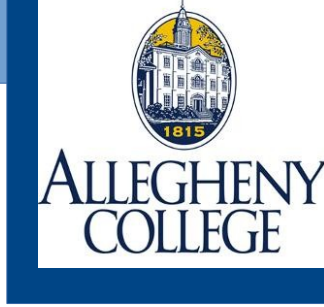
```
View(diamonds2)
# compare to the the size of original dataset
View(diamonds)
# Note: Good data may have been lost by dropping rows.
```

IfElse(): Condition Statement

```
y = ifelse(y < 3 | y > 20, NA, y)
```

- **Function**
- **Test Condition**
- **If True, then assign this**
- **If False, then assign this**

Data: *Diamond*



The book recommends to *mark* the data as bad or missing.

diamonds2 <- diamonds %>%

  mutate(y = **ifelse(y < 3 | y > 20, NA, y)**)

syntax: `ifelse(test, yes, no)`

Inspect each value of *y.* If the *y* is not between 3 and 20, then *y* = NA, else *y* = *y*

We Plot All Non-NA Values

```
# Missing, outliers values marked as NA
ggplot(data = diamonds2, mapping =
aes(x = x, y = y)) +  geom_point()
# compared to, no removed missing or
outlier values
ggplot(data = diamonds, mapping =
aes(x = x, y = y)) +  geom_point()
```

Missing Values, continued

```
# remove the outliers for y (i.e., y<3 and y >20)
library(tidyverse)
?ifelse # get online help
diamonds2 <- diamonds %>%
    mutate(y_between3And20 =
    ifelse(y < 3 | y > 20, NA, y))
ggplot(data = diamonds2,
   mapping = aes(x = x, y = y)) +  geom_point()
```

Add the ifelse() code directly to your other code.

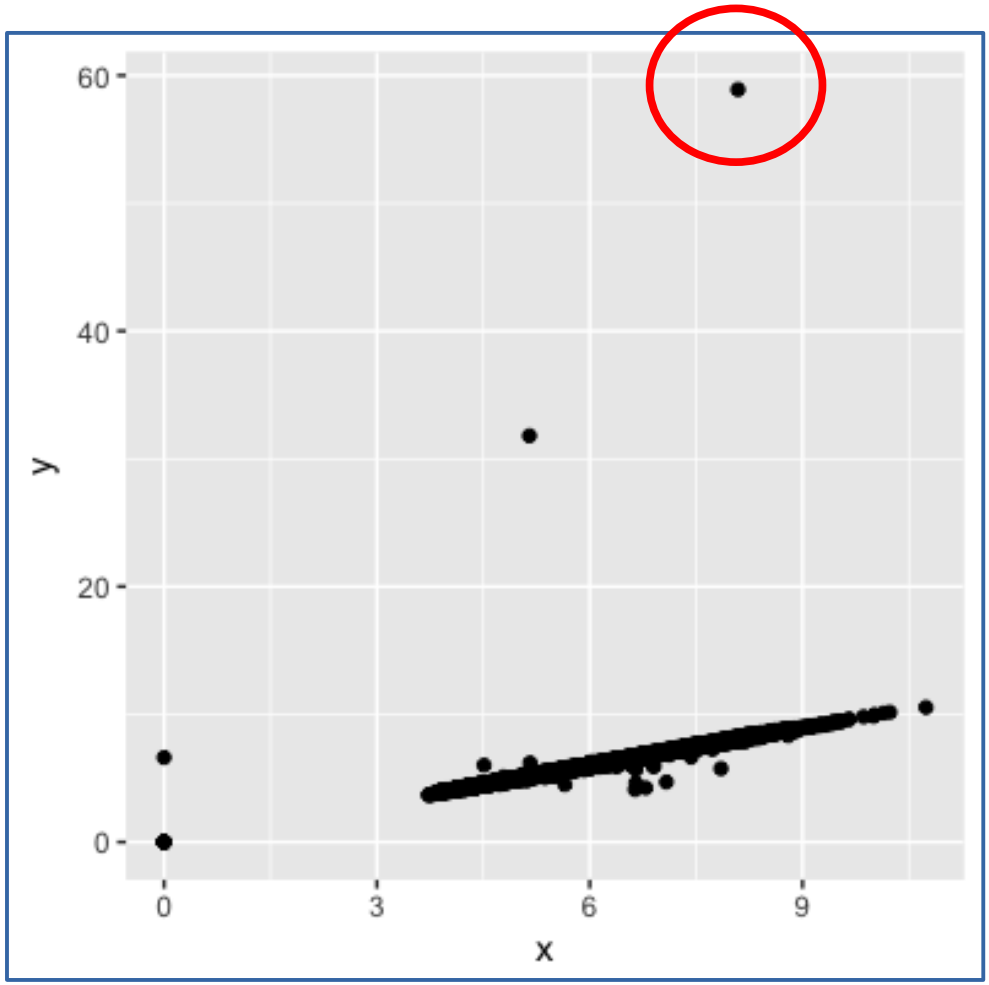
Missing Values, continued

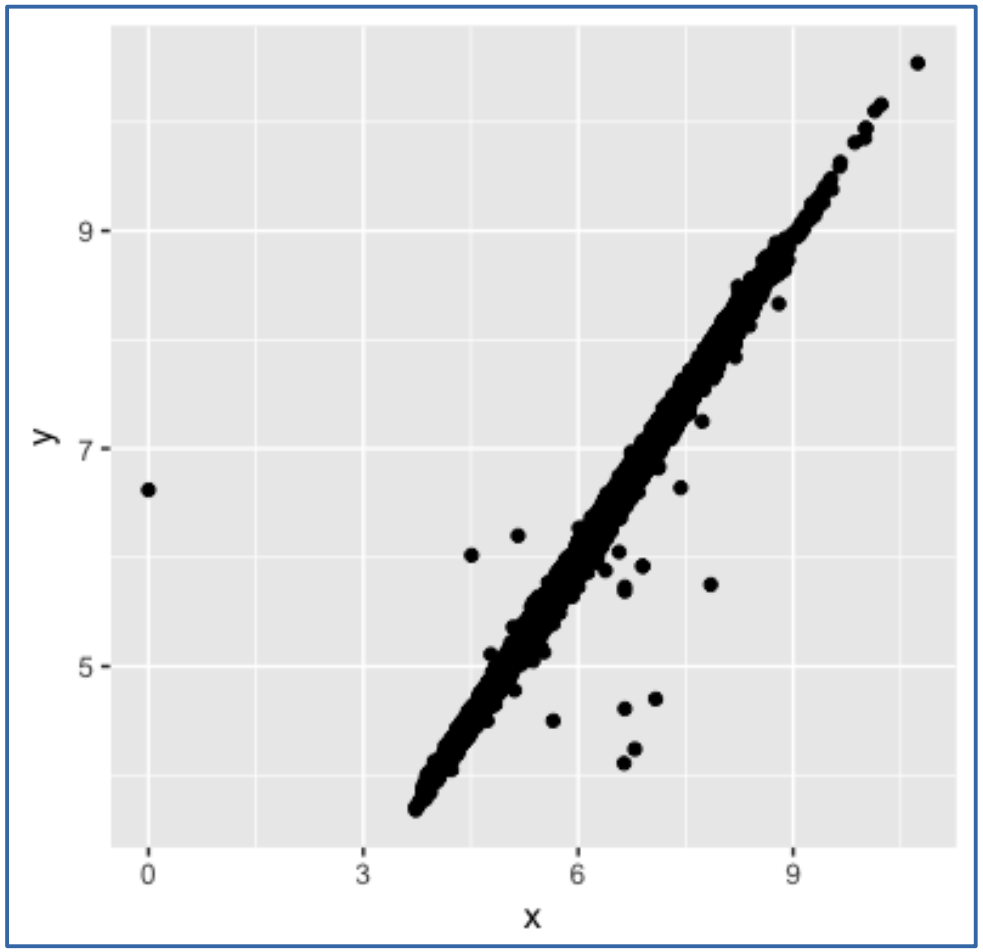
Throw out all values less than 3 and greater than 20 (i.e., y<3 and y >20)

	carat	cut	color	clarity	depth	table	price	x	y	z	y_between3And20
11964	1.00	Very Good	H	VS2	63.3	53	5139	0.00	0.00	0.00	NA
15952	1.14	Fair	G	VS1	57.5	67	6381	0.00	0.00	0.00	NA
24521	1.56	Ideal	G	VS2	62.2	54	12800	0.00	0.00	0.00	NA
26244	1.20	Premium	D	VVS1	62.1	59	15686	0.00	0.00	0.00	NA
27430	2.25	Premium	H	SI2	62.8	59	18034	0.00	0.00	0.00	NA
49557	0.71	Good	F	SI2	64.1	60	2130	0.00	0.00	0.00	NA
49558	0.71	Good	F	SI2	64.1	60	2130	0.00	0.00	0.00	NA
31601	0.20	Premium	D	VS2	62.3	60	367	3.73	3.68	2.31	3.68
31597	0.20	Premium	F	VS2	62.6	59	367	3.73	3.71	2.33	3.71
31599	0.20	Very Good	E	VS2	63.4	59	367	3.74	3.71	2.36	3.71
31602	0.20	Premium	D	VS2	61.7	60	367	3.77	3.72	2.31	3.72

	carat	cut	color	clarity	depth	table	price	x	y	z	y_between3And20
24068	2.00	Premium	H	SI2	58.9	57	12210	8.09	58.90	8.06	NA
49190	0.51	Ideal	E	VS1	61.8	55	2075	5.15	31.80	5.12	NA
27416	5.01	Fair	J	I1	65.5	59	18018	10.74	10.54	6.98	10.54

Trimmed Data, Slightly Different Plot...





- Left: WITH outliers
- Above: NO outliers

Data: *Diamond*

Can you use the below code to further trim outliers or missing data?

Plot your new graphic after using *ifelse()*

```
diamonds3 <- diamonds %>%
  mutate(y = ifelse(y < ## | y > ##, NA, y))
```

Missing Values May Have Their Own Meanings

- Q: Does missing flight arrival-time data indicate canceled flights?

Missing Values May Have Their Own Meanings

```
# install the flights data, if necessary.
#install.packages("nycflights13")
library(tidyverse, nycflights13)
flights <- nycflights13::flights
View(flights)


# Where are the missing values
flights$dep_time
```

The Distribution of a **Continuous** Variable, Aggregated By a **Categorical** variable

```
# Compare the scheduled departure times
for cancelled and non-cancelled times
flights %>%
  mutate(
    cancelled = is.na(dep_time),
    #%/% is a whole number division
    sched_hour = sched_dep_time %/% 100,
    sched_min = sched_dep_time %% 100,
    sched_dep_time = sched_hour + sched_min / 60
  ) %>%
  ggplot(mapping = aes(sched_dep_time)) +
  geom_freqpoly(mapping = aes(colour = cancelled),
  binwidth = 1/4)
```

We compare the scheduled departure times between cancelled and non-cancelled flights using a frequency polygon plot

Erm, What Did That Previous Code Do?

First, the data frame `flights` is being piped using the %>% operator to allows to transform for the next step.

1. **Data source**:

`flights %>%`

Start with the flights dataset.

2. **Make a new column**:

`mutate(cancelled = is.na(dep_time),`

A new column cancelled is created:

It is TRUE if `dep_time` is NA → the flight was cancelled.

It is FALSE if `dep_time` is not NA → the flight departed.

Erm, What Did That Previous Code Do?

3.Break up the time to prepare for plotting:

```
sched_hour = sched_dep_time %/% 100,
sched_min = sched_dep_time %% 100,
```

`sched_dep_time` is in HHMM format (e.g., 530 for 5:30 AM).

`%/%` is integer division; returns the hour part.

`%%` is modulus; returns the minute part.

For example: Break up the time value into *hours* and *minutes*.

```
sched_dep_time = 530
sched_hour = 5 # hours
sched_min = 30 # minutes
```

So, similarly,

```
sched_dep_time = sched_hour + sched_min / 60
```

Erm, What Did That Previous Code Do?

4. Plotting:

```
ggplot(mapping = aes(sched_dep_time)) +
  geom_freqpoly(mapping = aes(colour = cancelled), binwidth = 1/4)
```

Creates a frequency polygon (similar to a histogram, but with lines instead of bars).

`aes(sched_dep_time)`

maps the x-axis to scheduled departure time in hours.

`aes(colour = cancelled)`

colors the lines differently for cancelled and non-cancelled flights.

`binwidth = 1/4`

means each bin covers 15 minutes (0.25 hours).

Erm, What Did That Previous Code Do?

5. **Conclusions**:

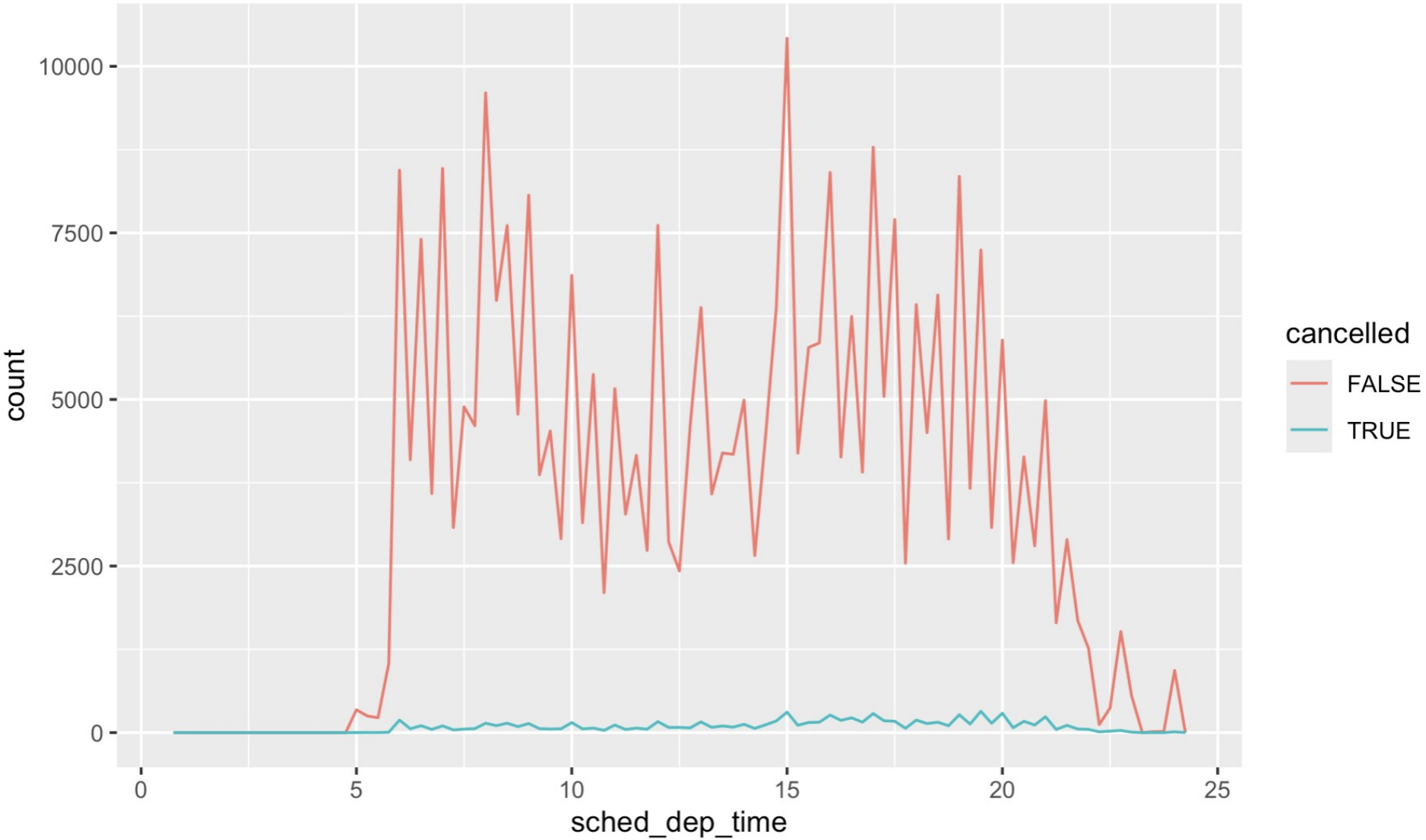
So what? This code visualizes when during the day flights are more likely to be cancelled.

It shows the distribution of scheduled departure times, comparing cancelled vs non-cancelled flights in a clear, time-based plot.

We address questions like:

- *Are cancellations more common in the morning, afternoon, or evening?*
- *Is there a time of day with significantly more cancellations?*

Flight Cancellations Plot

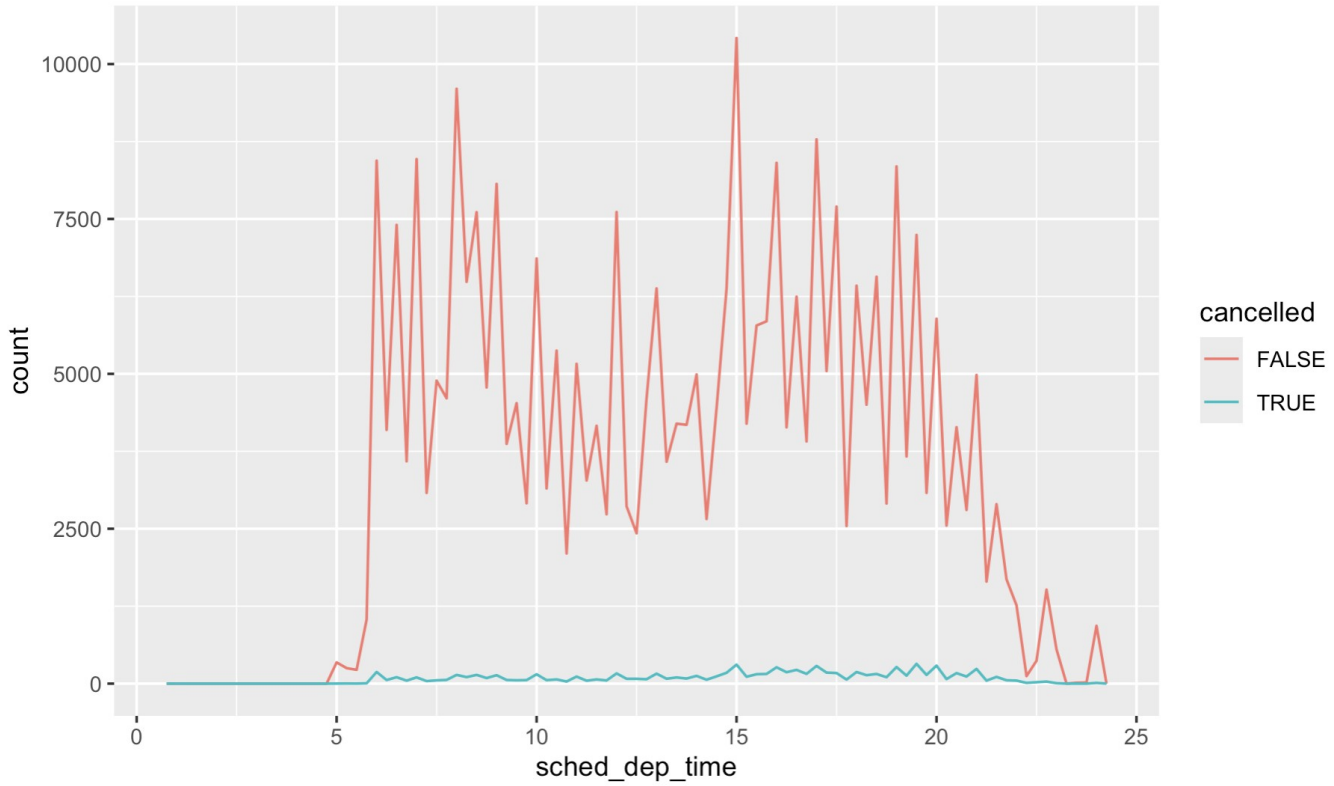


Flight Cancellations Plot

Binwidth: 15 minute intervals

Y axis

Count: The number of flights scheduled at that time, binned into 15-minute intervals (binwidth = 0.25 hours).

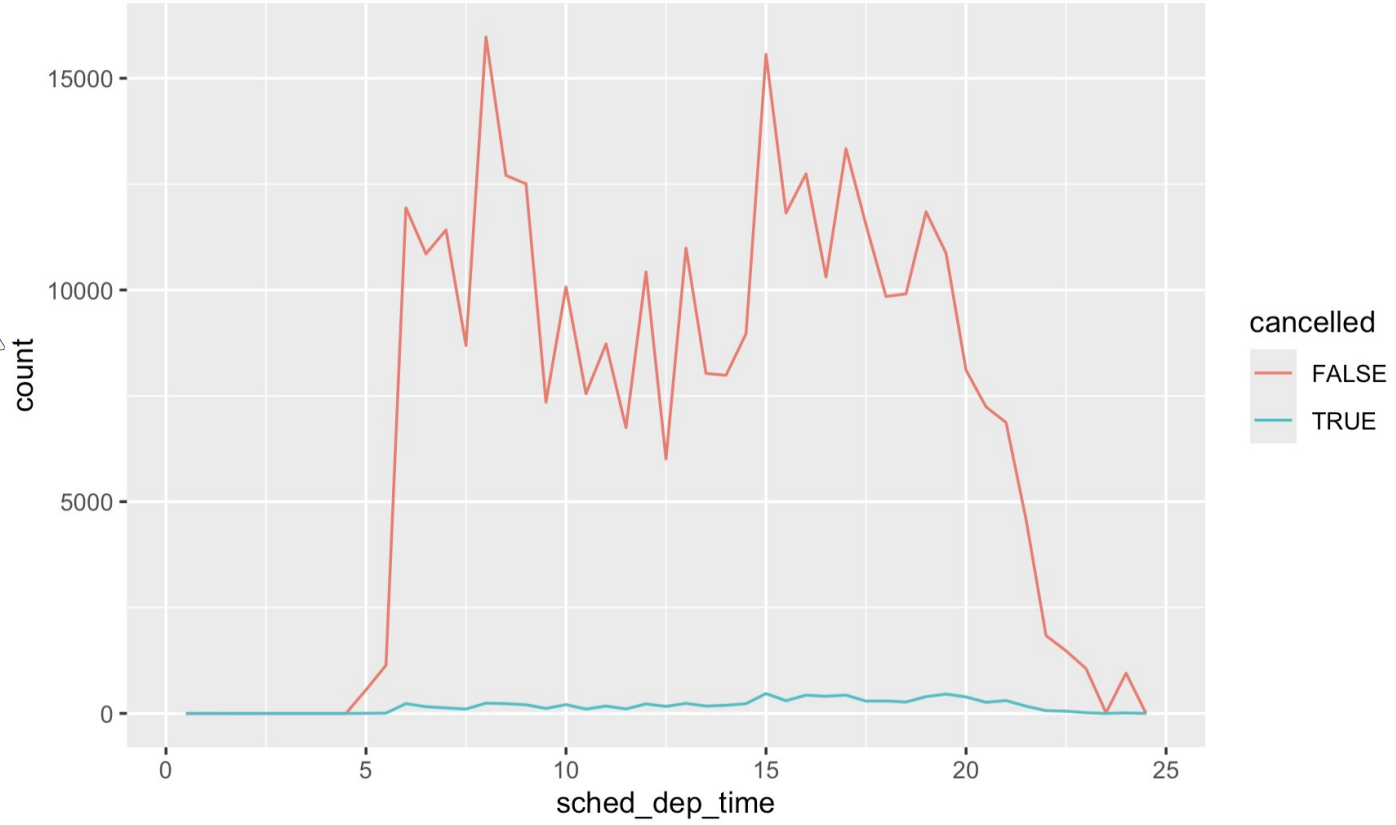


Flight Cancellations Plot

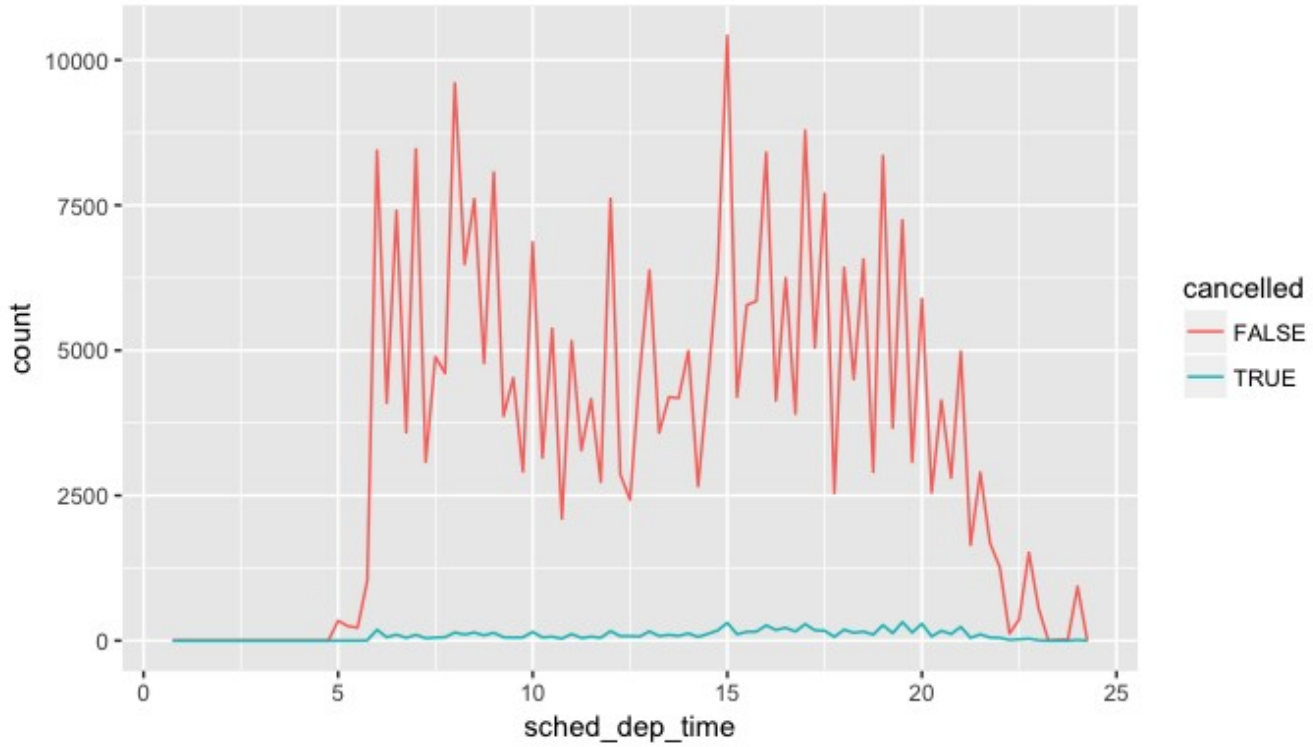
Binwidth: 30 minute intervals

Y axis

Count: The number of flights scheduled at that time, binned into 30-minute intervals (binwidth = 0.50 hours).



Conclusions



Each line shows how many flights were scheduled at a given time of day.

So, comparing the heights of the two lines at each time gives insight into:

- When flights are more likely to be scheduled (red line)
- When flights are more likely to be cancelled (green line)

Covariation

covariance

co·var·i·ance

/ˌkōˈverēəns/

noun

1. MATHEMATICS
   the property of a function of retaining its form when the variables are linearly transformed.
2. STATISTICS
   the mean value of the product of the deviations of two variates from their respective means.

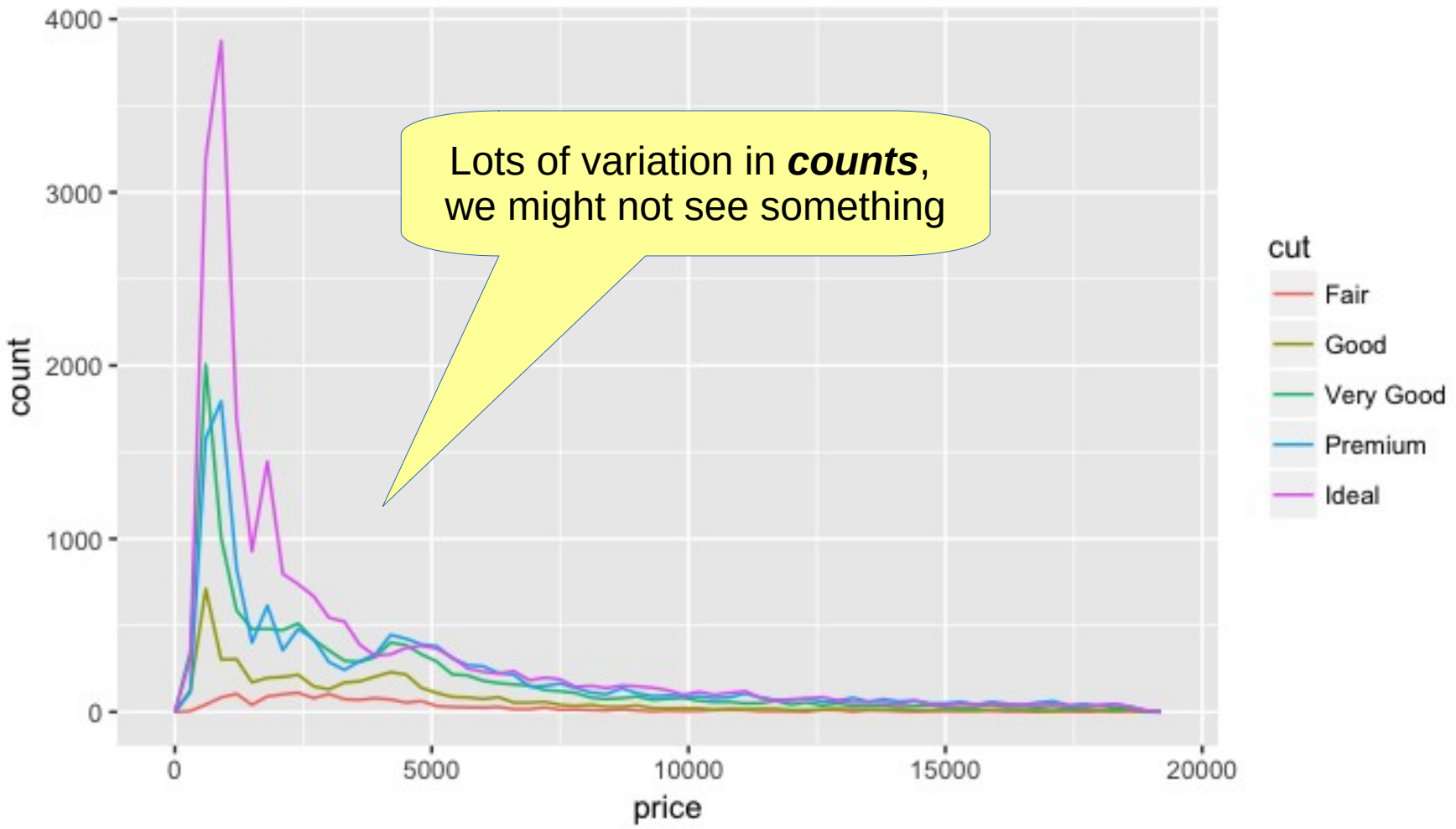
- **Covariation** is the tendency for the values of two or more variables to vary together in a related way.
- Study covariation by visualizing relationships between two or more variables.
- Pay attention to your variables to known how best to visualize these variables

Covariation

- Back to the Diamonds dataset
- How do the prices of diamonds vary with quality?

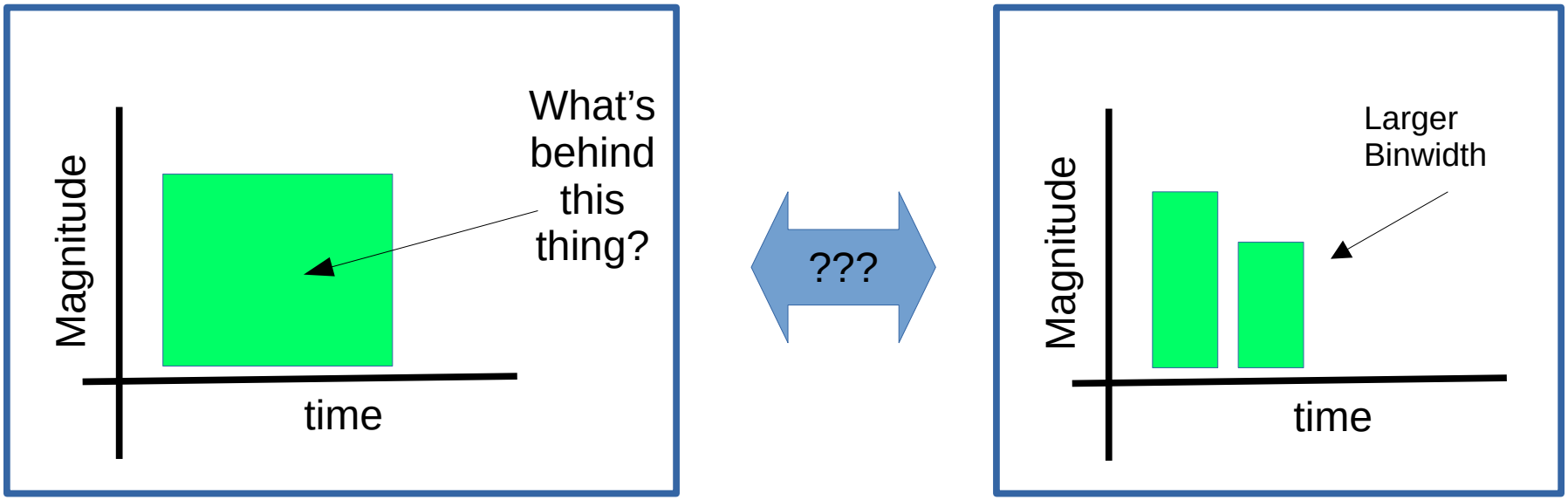
```
# Plot the count of each each cut quality according to price.

ggplot(data = diamonds, mapping = aes(x = price)) + geom_freqpoly(mapping = aes(colour = cut), binwidth = 500)
```

The Plot of the Diamond Counts



This Plot May Make It Hard To See The Phenomenon

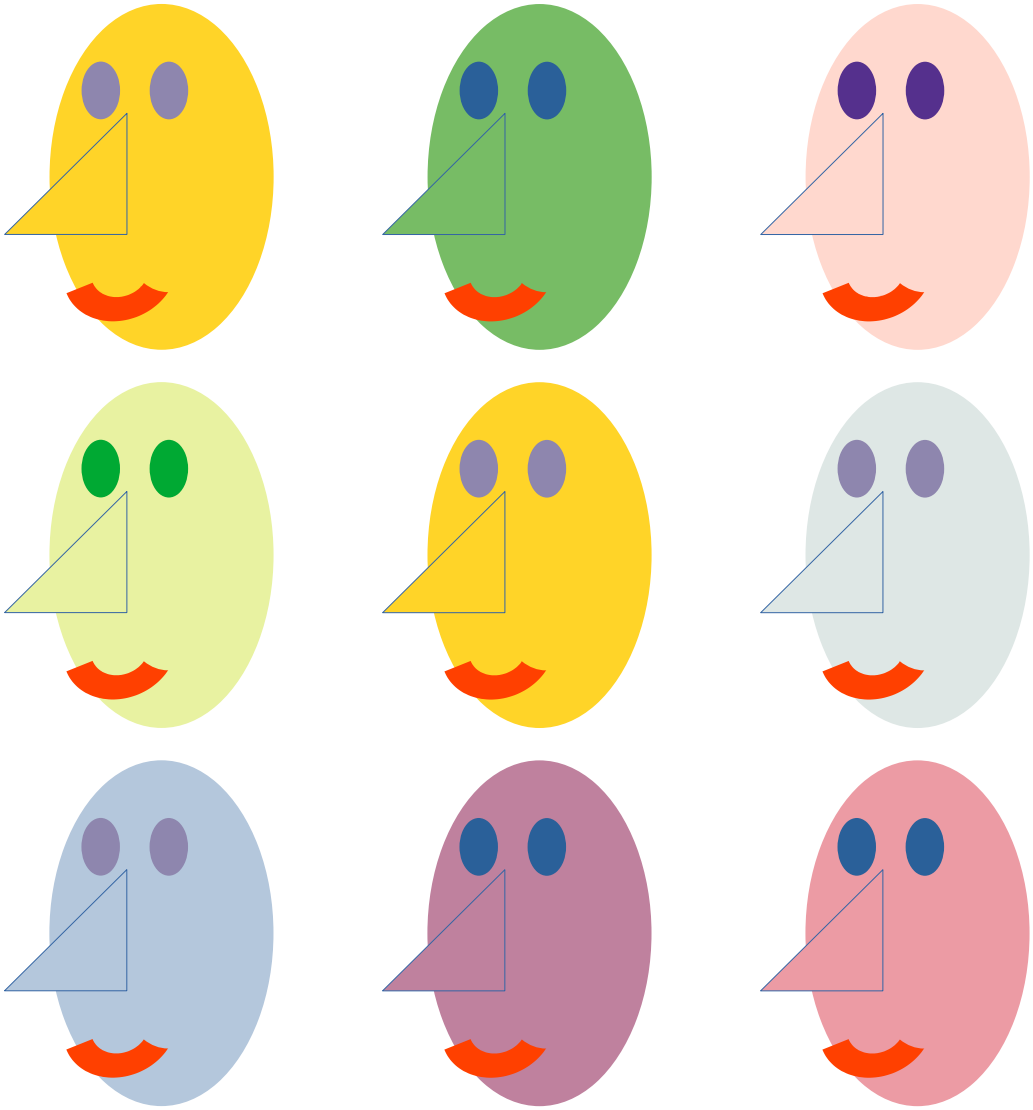
- The counts variable seems to have values from all over the range.
- This is noise in our plot
- If one group is much smaller than the others, then it is hard to see the differences in its distribution.



Hey, How to Think About Binwidths?

Hey Nine Friends! Come try my fresh pizza that I just made!

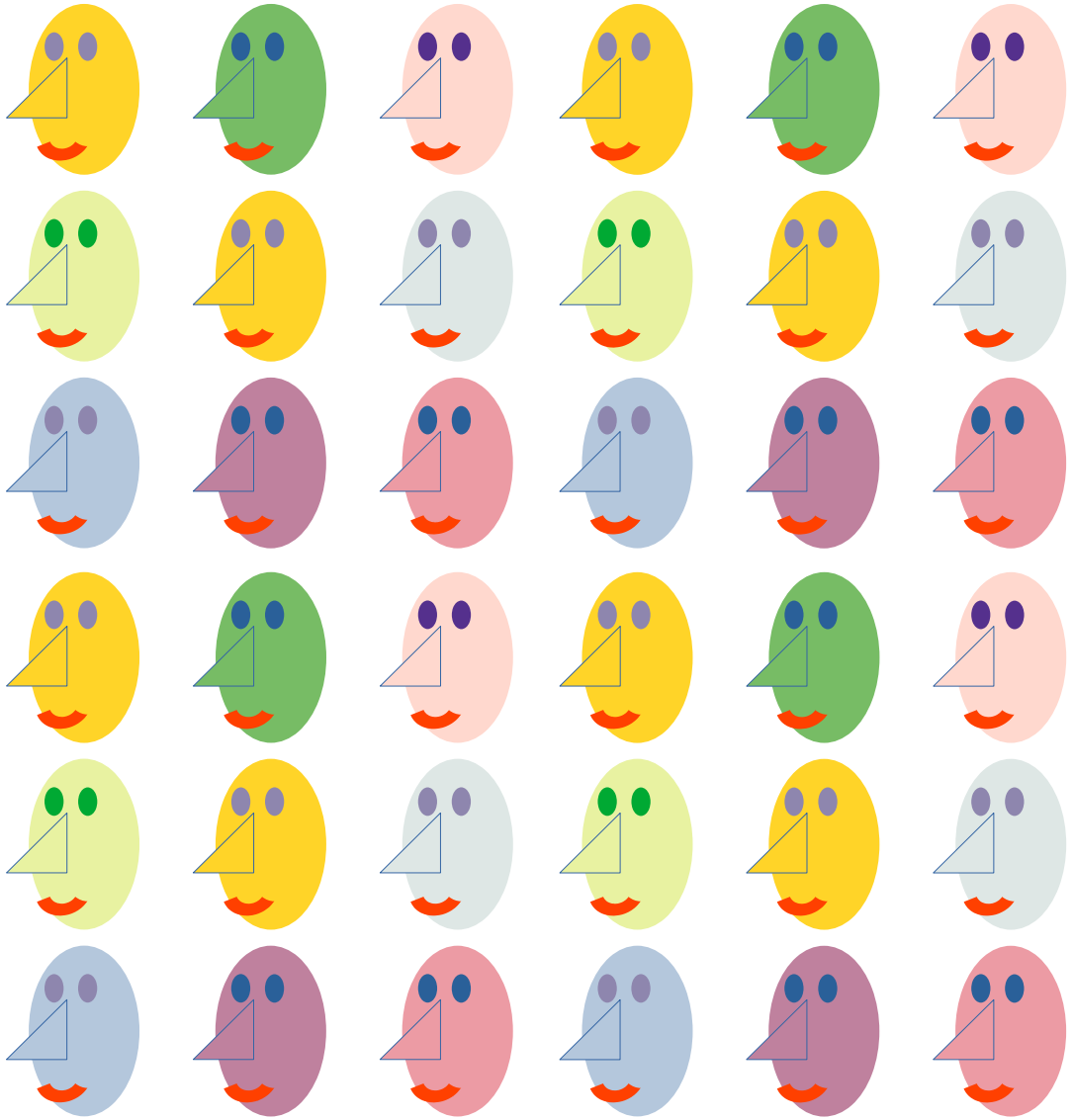
Each person gets a 1/9 size piece to try



Smaller Portions?

Hey Thirty-Six Friends! Come try my fresh pizza that I just made!

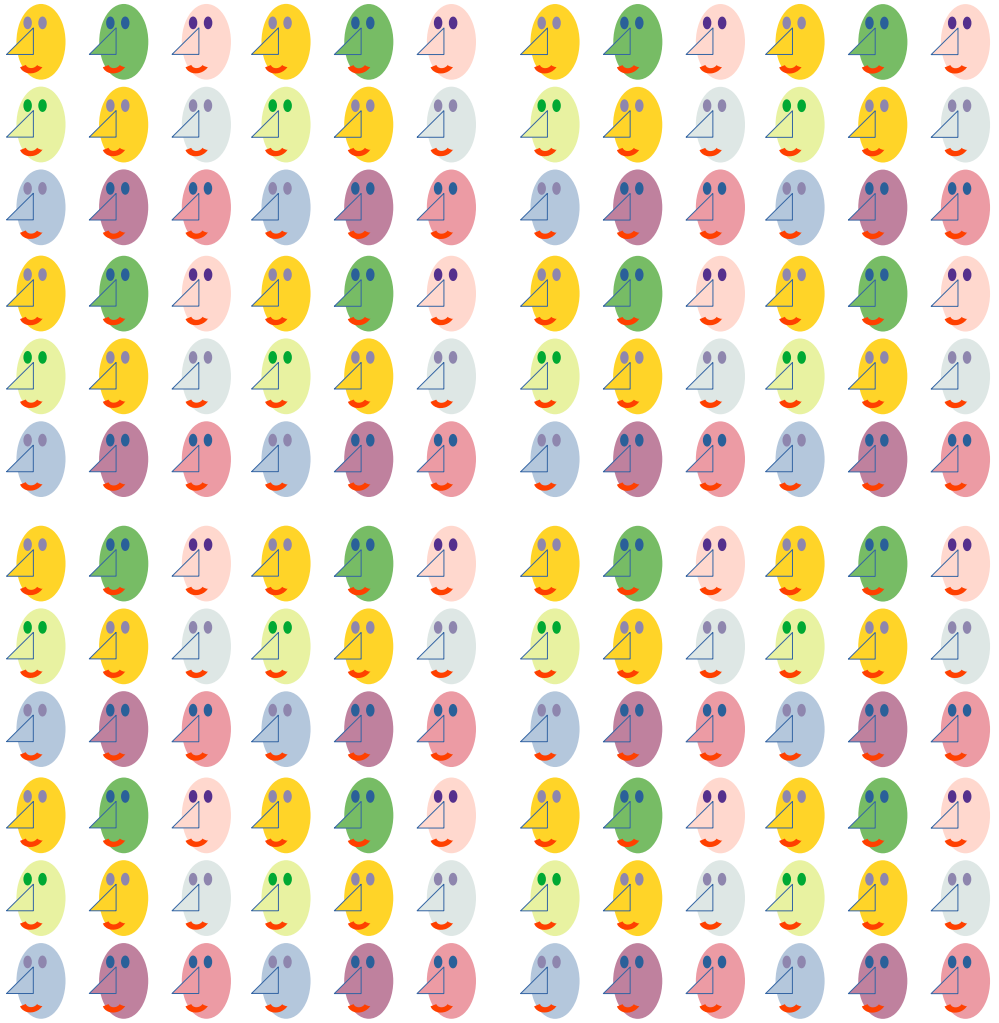
Each person gets a 1/36 sized piece to try



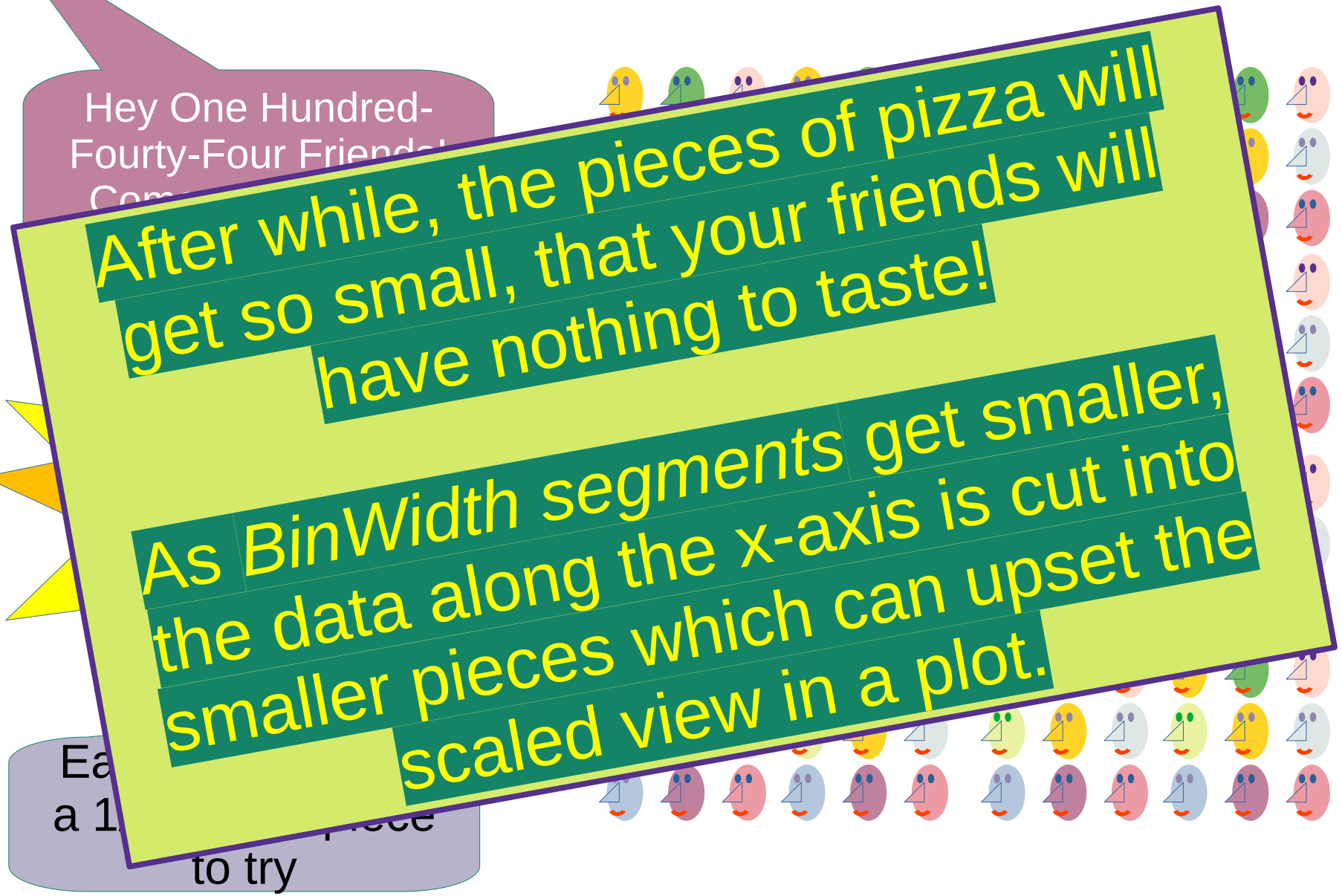
Even Smaller Portions?

Hey One Hundred-Fourty-Four Friends! Come try my fresh pizza that I just made!

Each person gets a 1/144 size piece to try



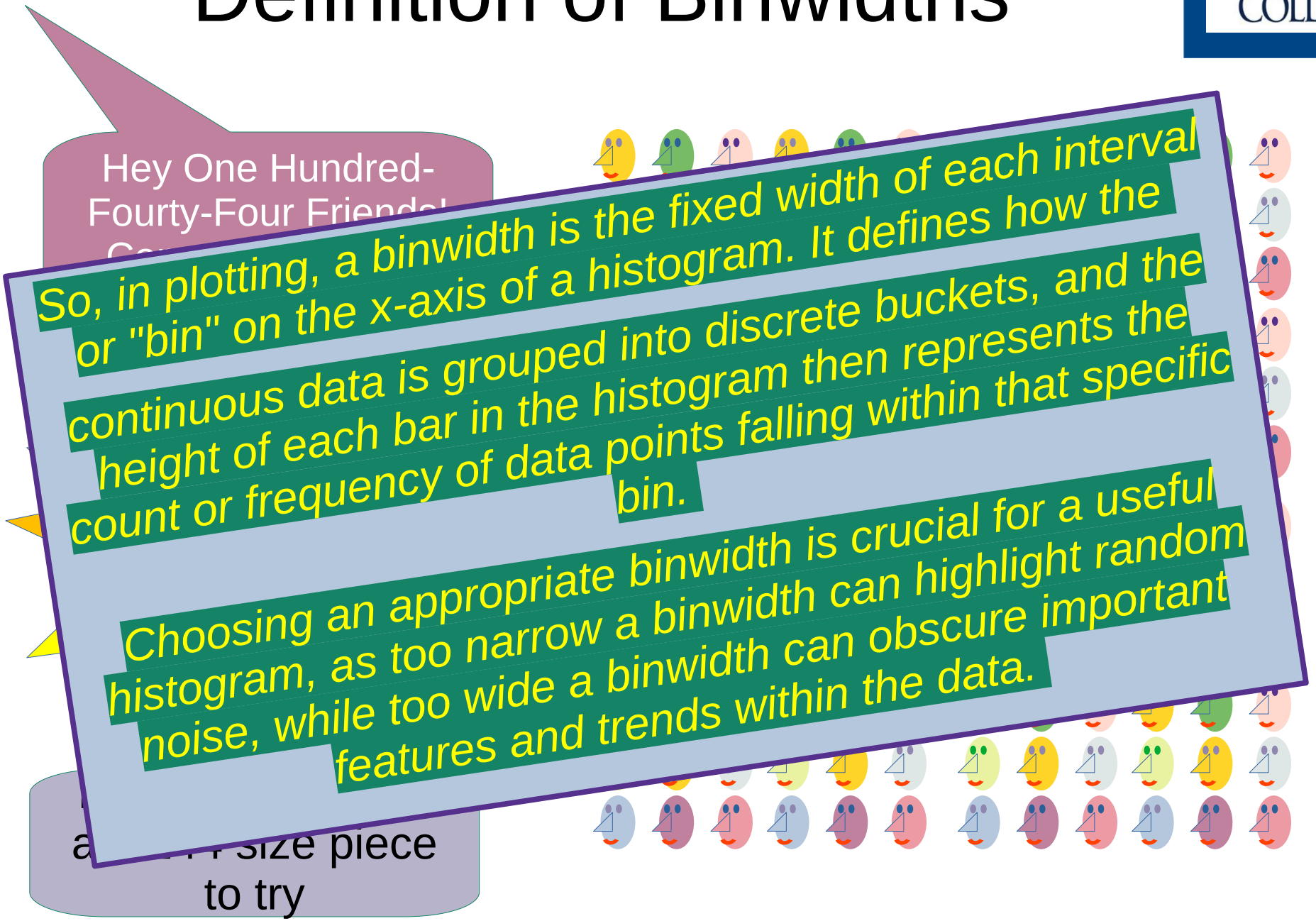
Binwidths?



After while, the pieces of pizza will get so small, that your friends will have nothing to taste!

As *BinWidth segments* get smaller, the data along the x-axis is cut into smaller pieces which can upset the scaled view in a plot.

Definition of Binwidths

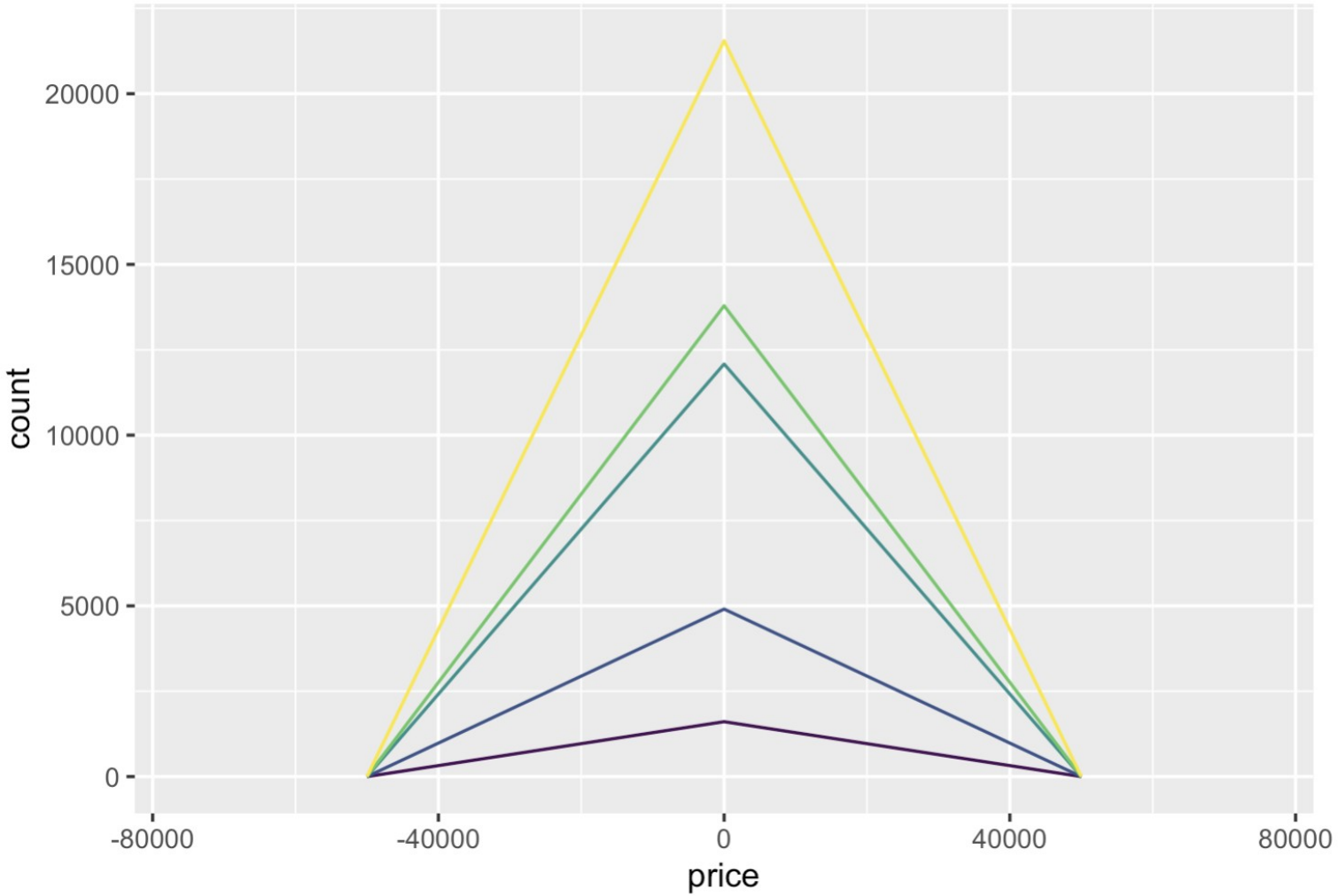


So, in plotting, a binwidth is the fixed width of each interval or "bin" on the x-axis of a histogram. It defines how the continuous data is grouped into discrete buckets, and the height of each bar in the histogram then represents the count or frequency of data points falling within that specific bin.

Choosing an appropriate binwidth is crucial for a useful histogram, as too narrow a binwidth can highlight random noise, while too wide a binwidth can obscure important features and trends within the data.

Comparing Binwidths

Data Divided into 5 Bins

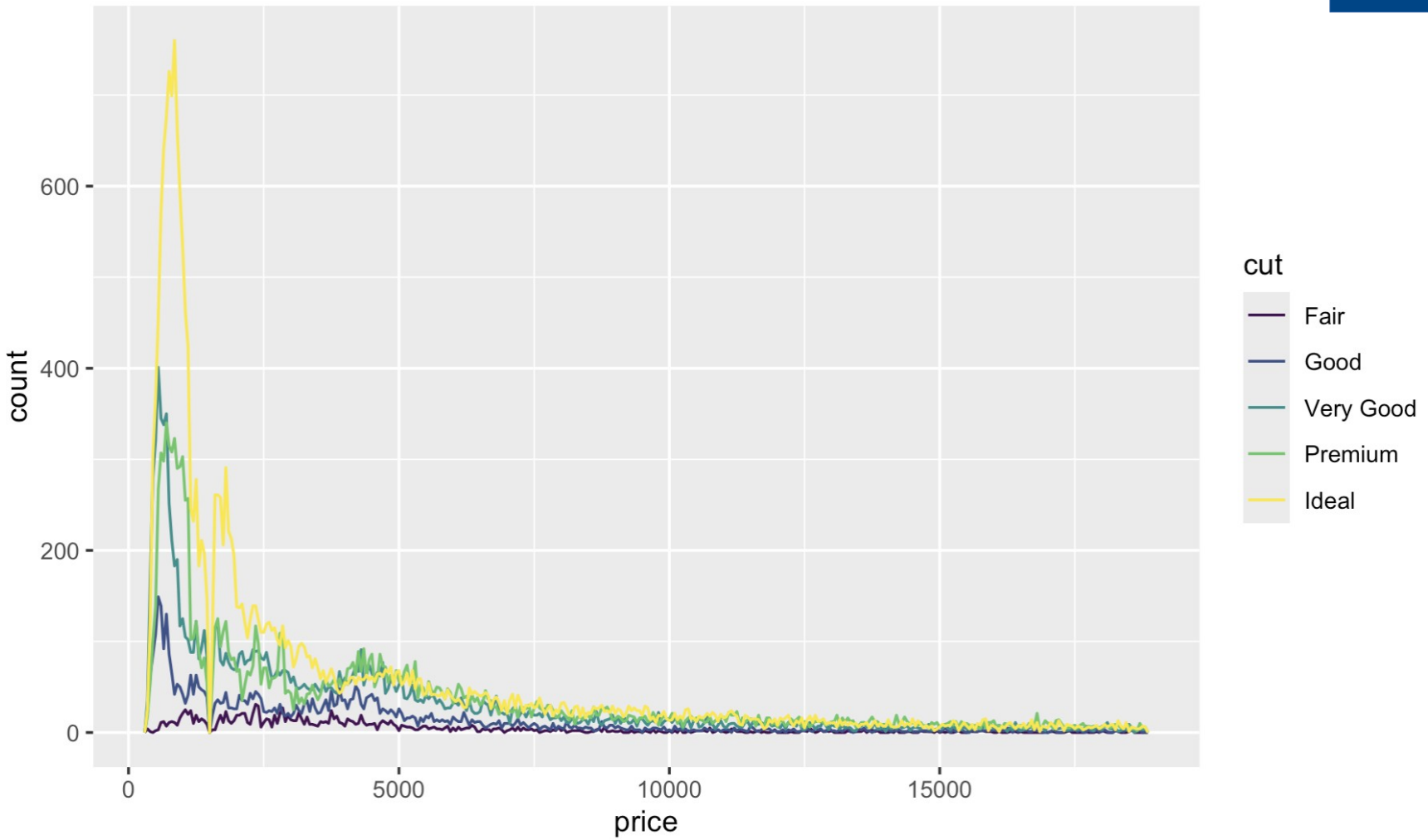


Huh?!

```
ggplot(data = diamonds, mapping = aes(x = price)) +
geom_freqpoly(mapping = aes(colour = cut), binwidth = 5)
```

Comparing Binwidths

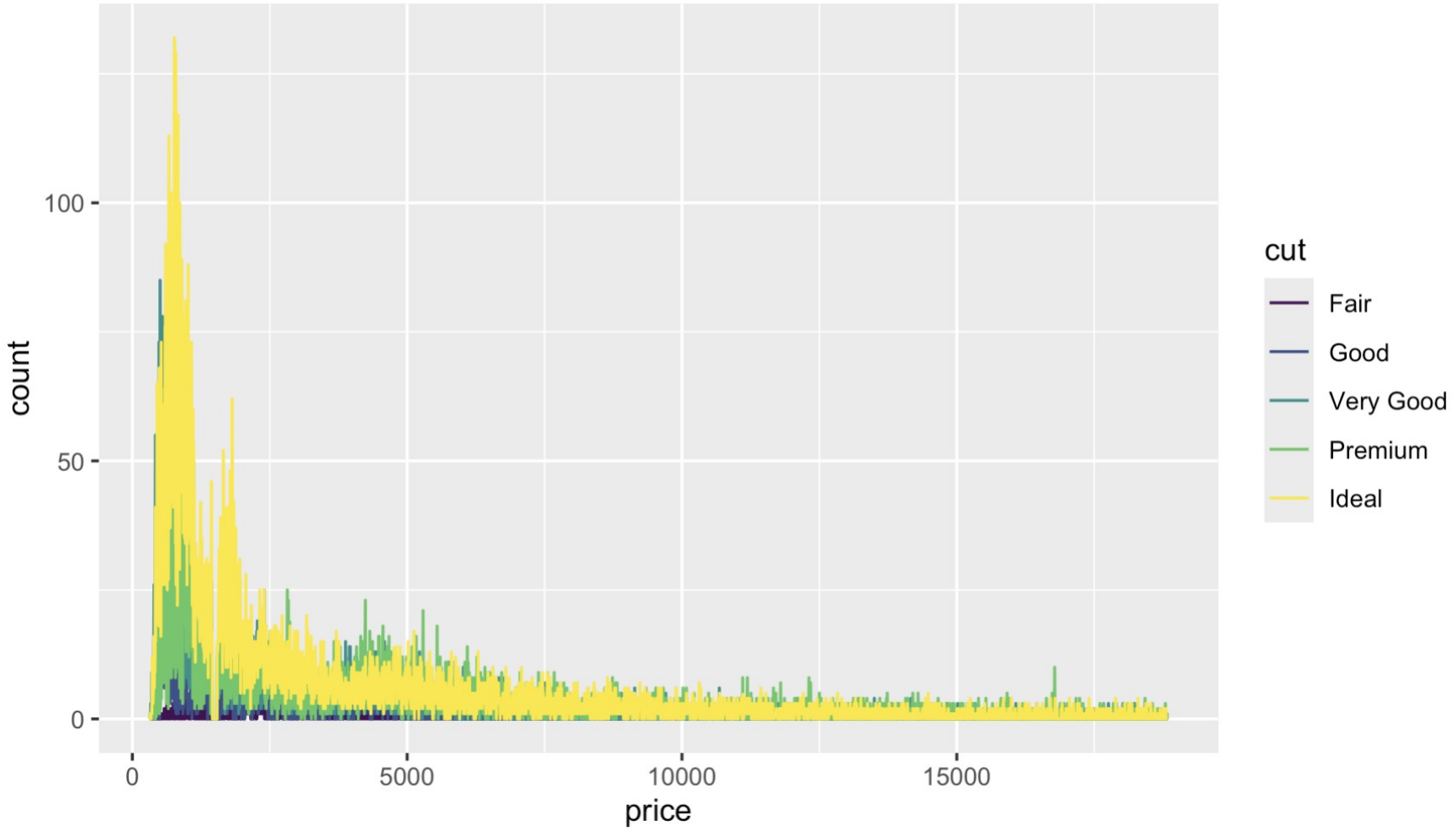
Data Divided into 50 Bins




```
ggplot(data = diamonds, mapping = aes(x = price)) +
geom_freqpoly(mapping = aes(colour = cut), binwidth = 50)
```

Comparing Binwidths

Data Divided into 50000 Bins



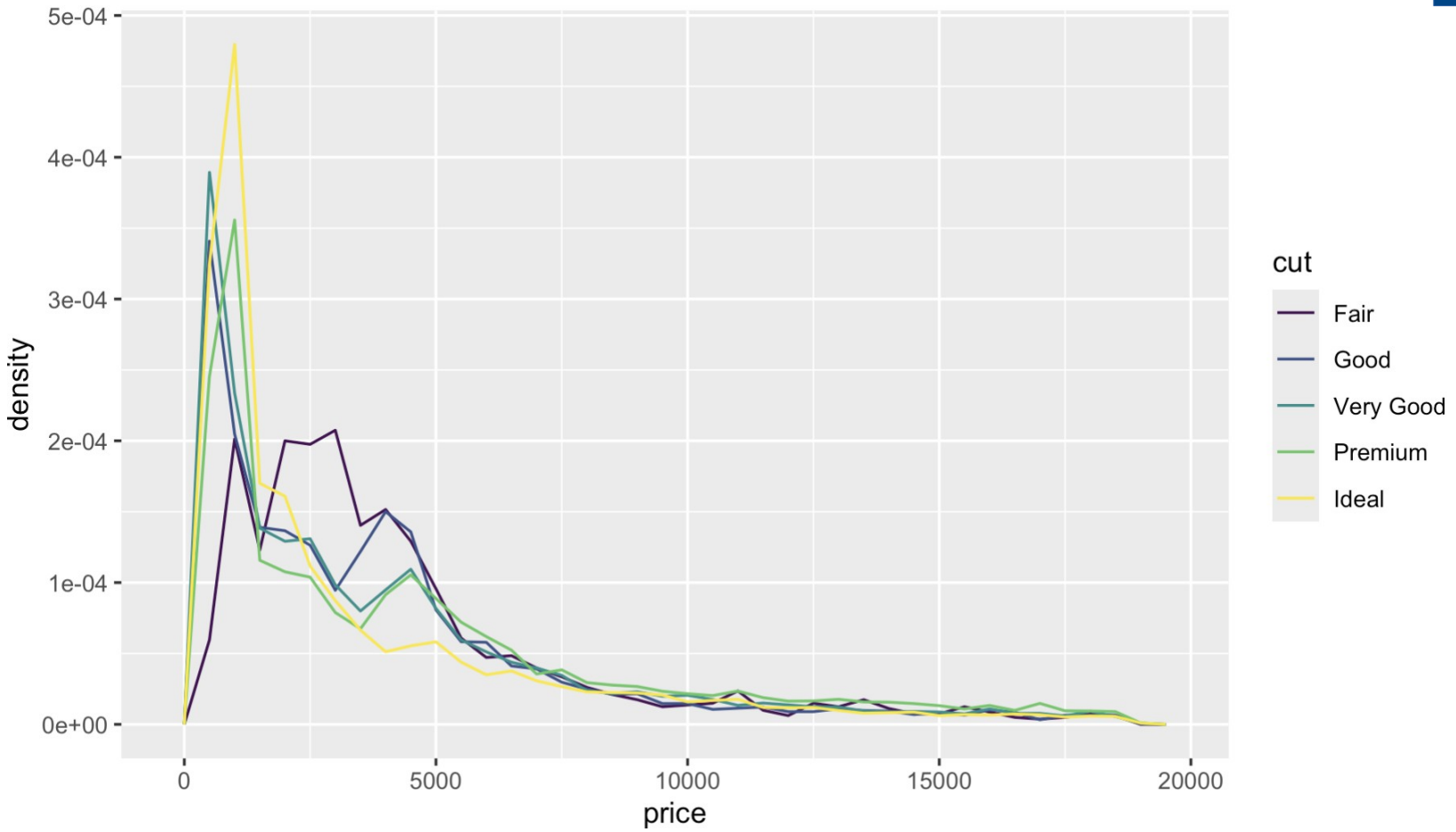

```
ggplot(data = diamonds, mapping = aes(x = price)) +
geom_freqpoly(mapping = aes(colour = cut), binwidth = 50000)
```

Let's Change Our Plotting

```
# Does a histogram help?
ggplot(diamonds) +  geom_bar(mapping = aes(x = cut))


#Note:  Density, the count is standardized so that the area under each frequency polygon is one unit
# We change the axis "level" the view for all
ggplot(data = diamonds, mapping = aes(x = price, y = ..density..)) +  geom_freqpoly(mapping = aes(colour = cut), binwidth = 500)
```

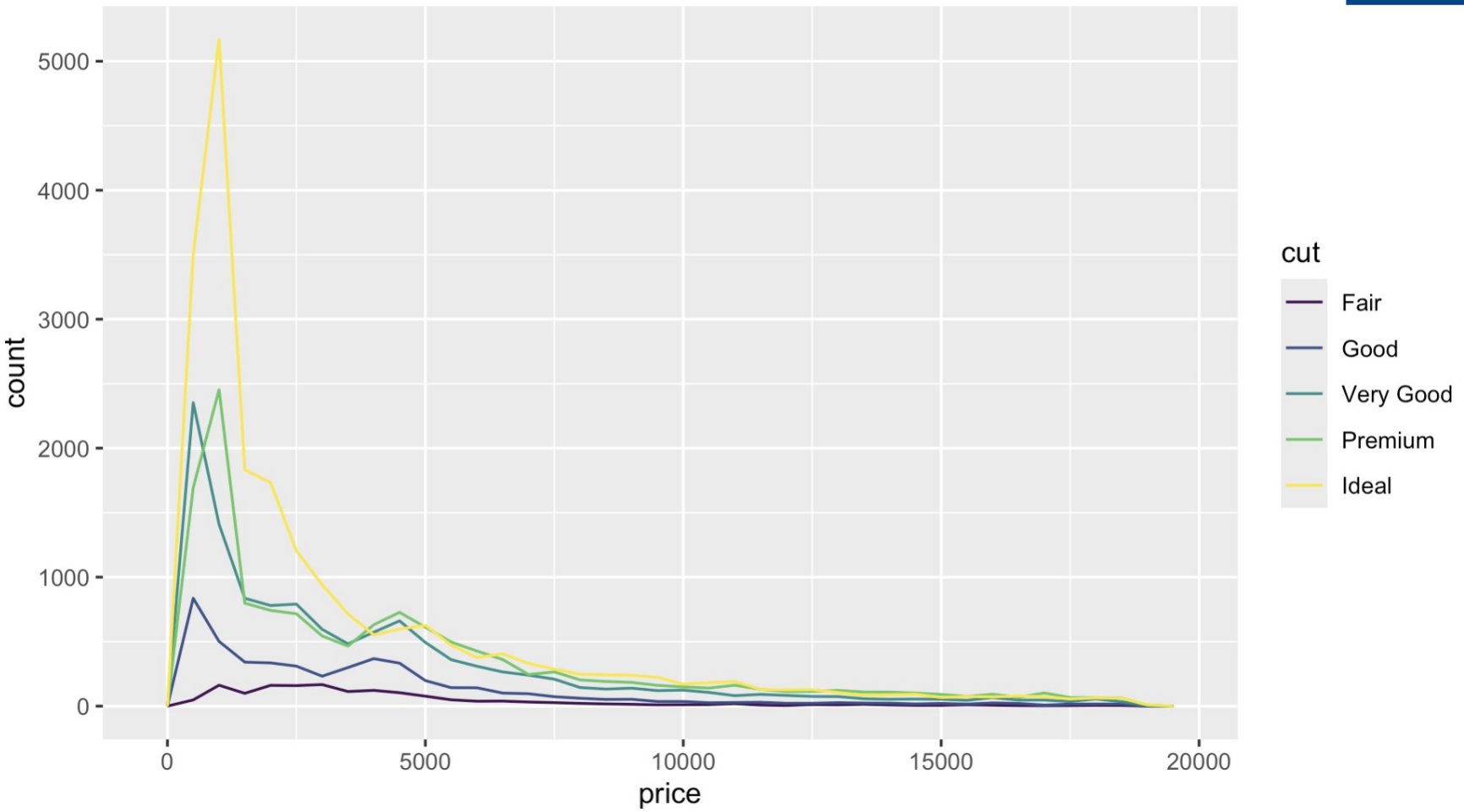
Density: To normalize your view!

Use a Density Plot?




```
# Does a line plot help?

ggplot(data = diamonds, mapping = aes(x = price, y = ..density..)) +  geom_freqpoly(mapping = aes(colour = cut), binwidth = 500)
```

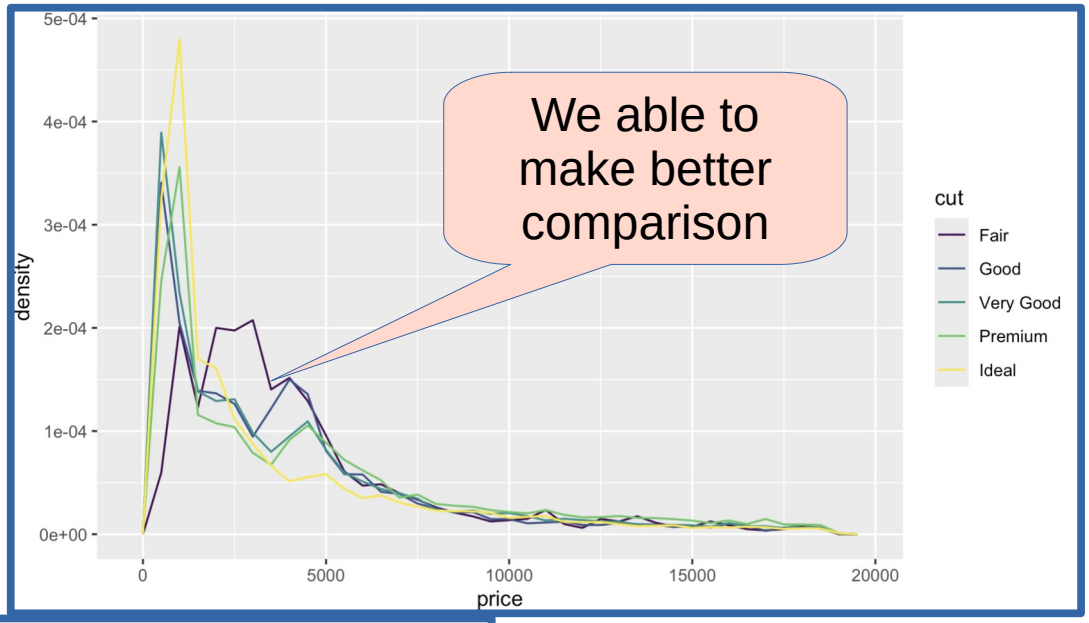
Or a Line Graph?

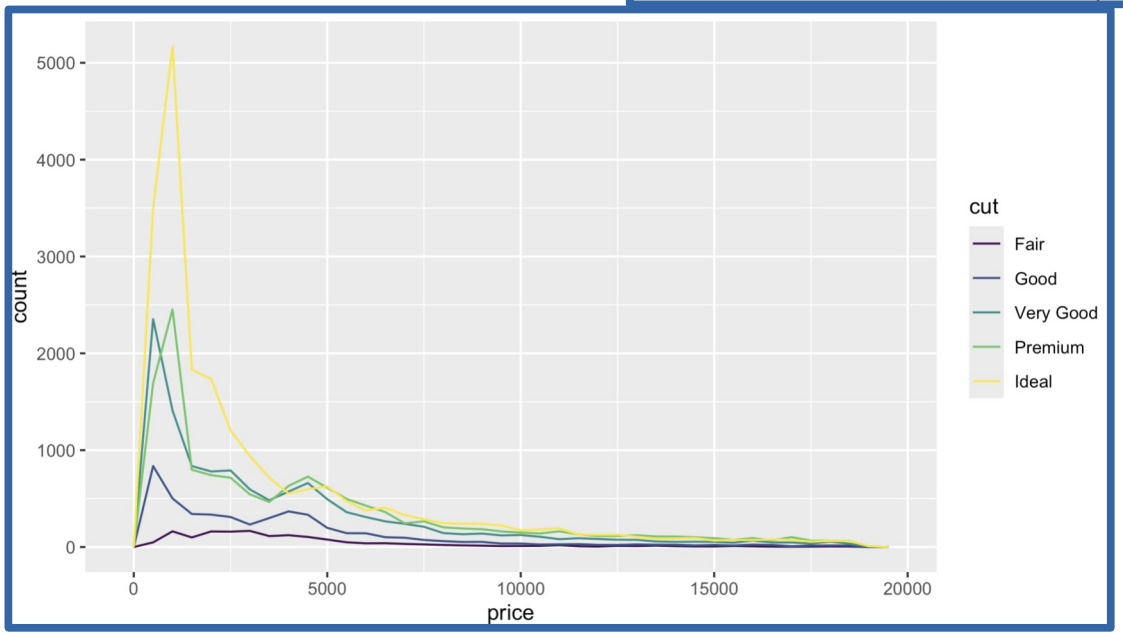



```
ggplot(data = diamonds, mapping = aes(x = price)) +
geom_freqpoly(mapping = aes(colour = cut), binwidth = 500)
```

Comparing Different Plots

```
mapping = aes(
   x = price,
   y = ..density..
 )
```

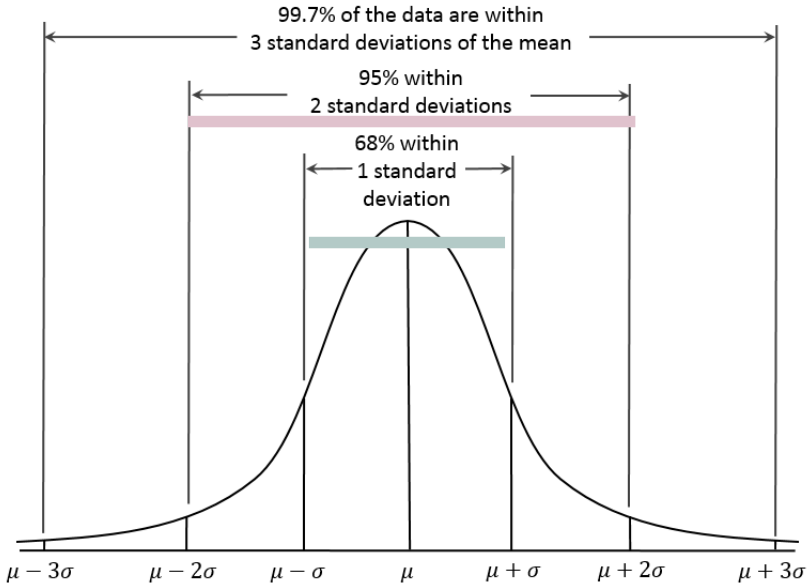


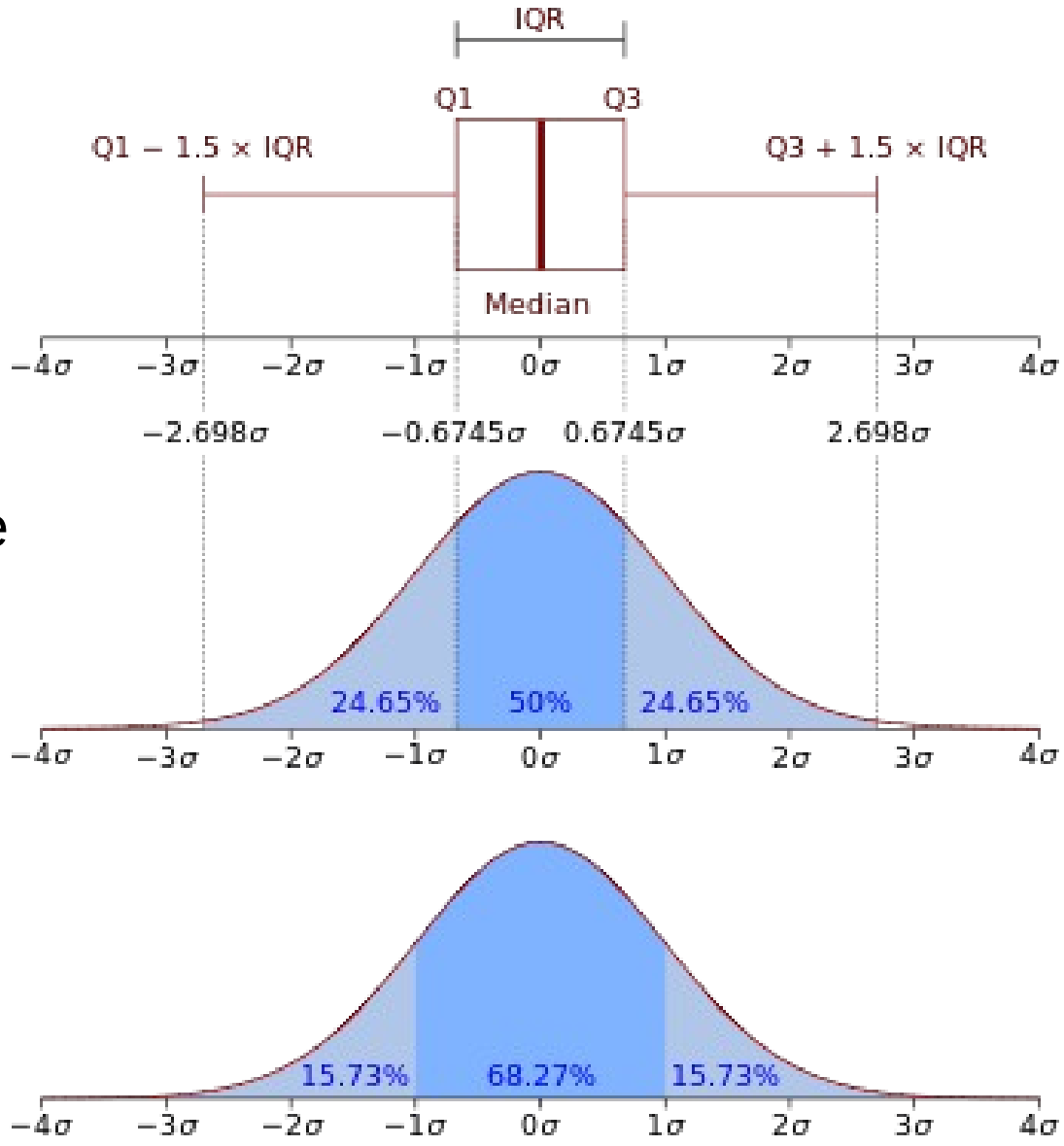



```
mapping = aes(
   x = price,
    )
```

Now, Let’s Talk Box Plots

Box Plots

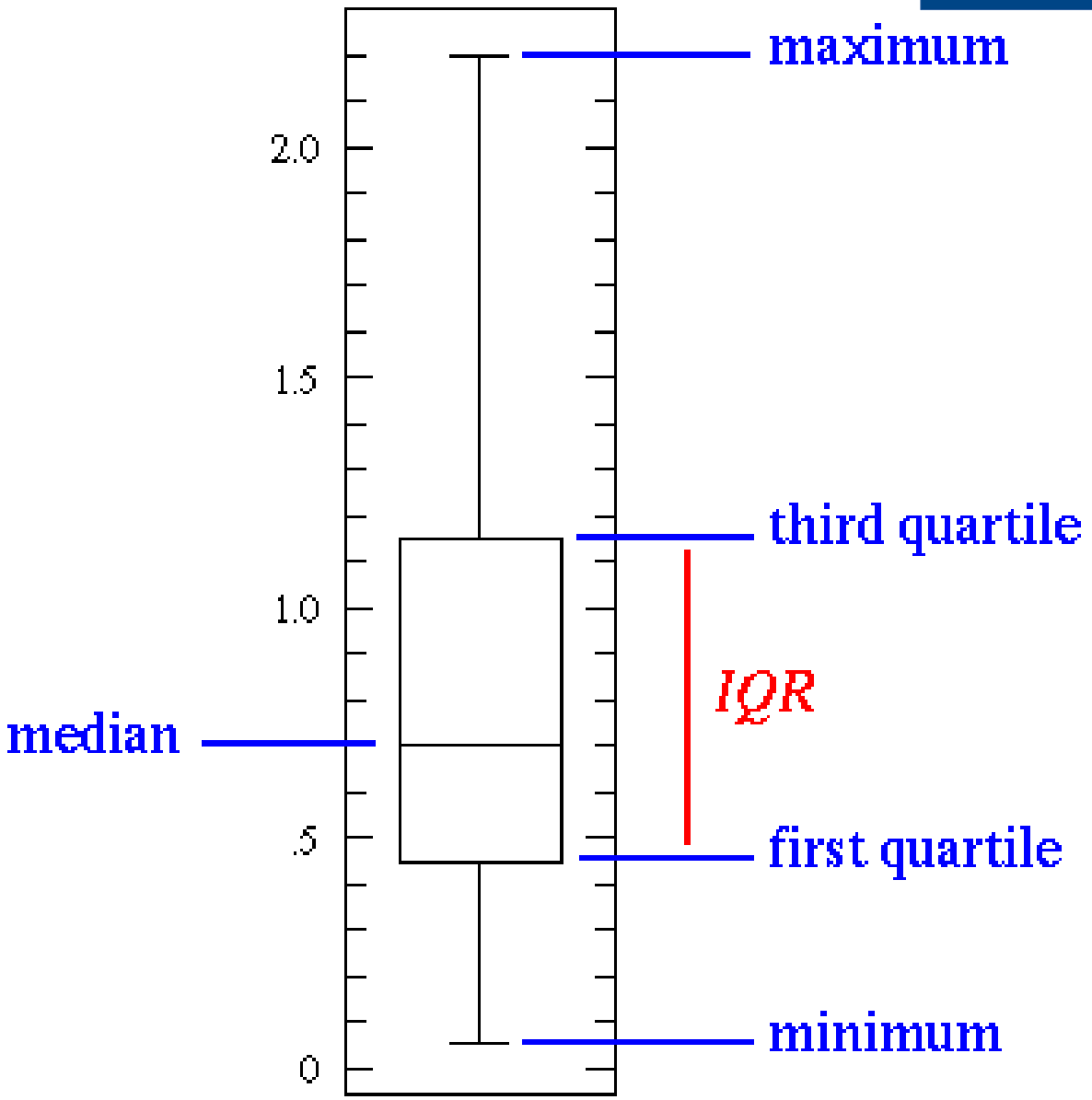




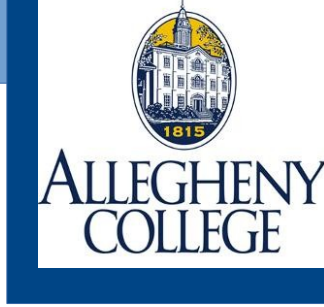
- For the Normal Distribution, the values less than one standard deviation away from the mean account for 68.27% of the set; while two standard deviations from the mean account for 95.45%; and three standard deviations account for 99.73%.

Explore Data Using Box Plots

Standardized way of displaying the distribution of data based on the five number summary: minimum, first quartile, median, third quartile, and maximum

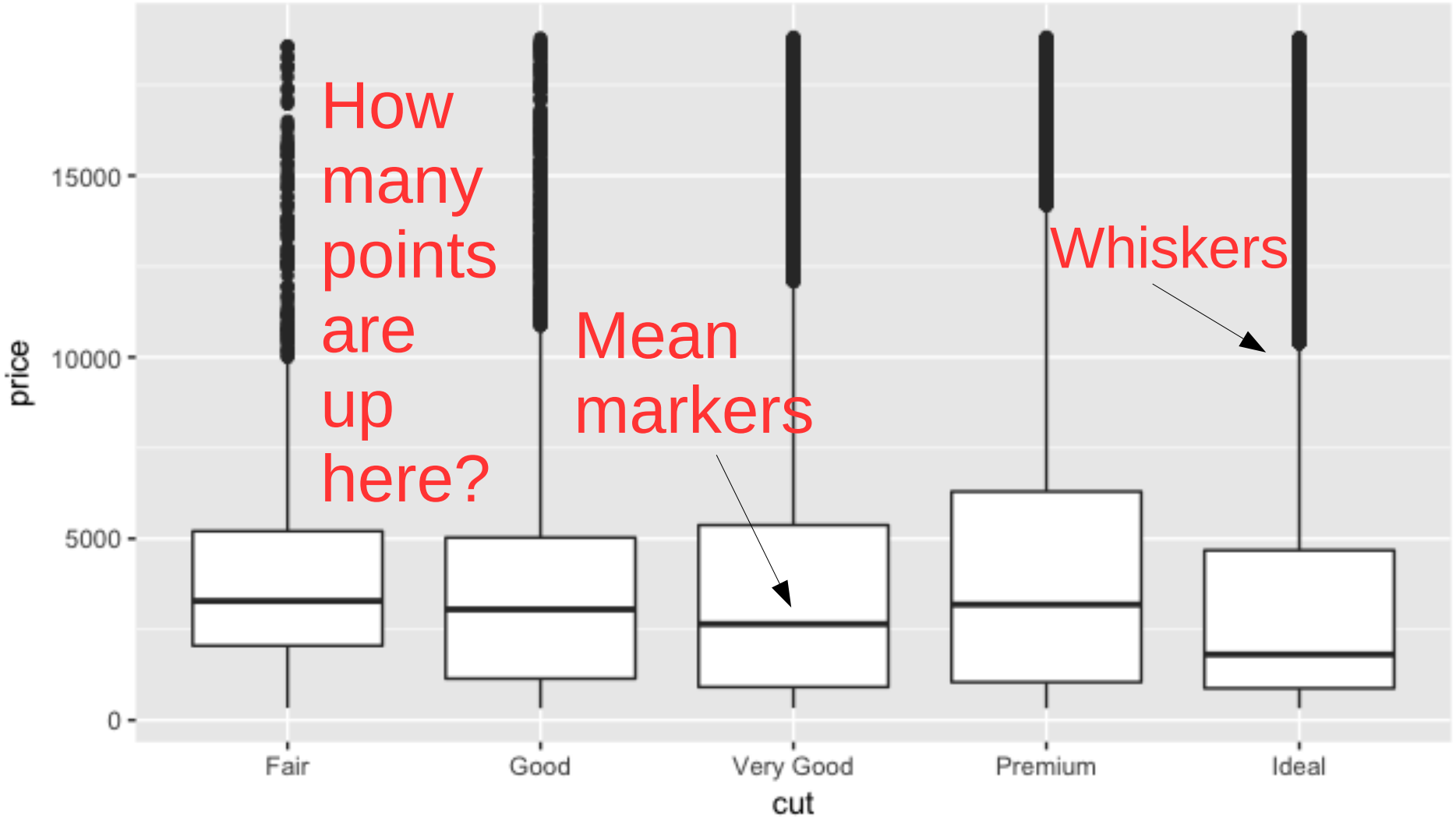


Explore Data Using Box Plots




```
# Make a box plot to describe covariance between cut and price.

ggplot(data = diamonds, mapping = aes(x = cut, y = price)) + geom_boxplot()
```

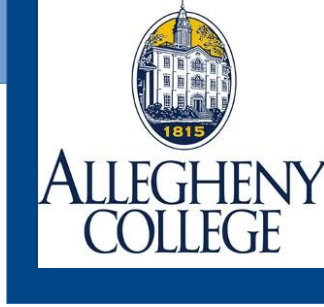
Explore Data Using Box Plots



Box Plots: Pros and Cons

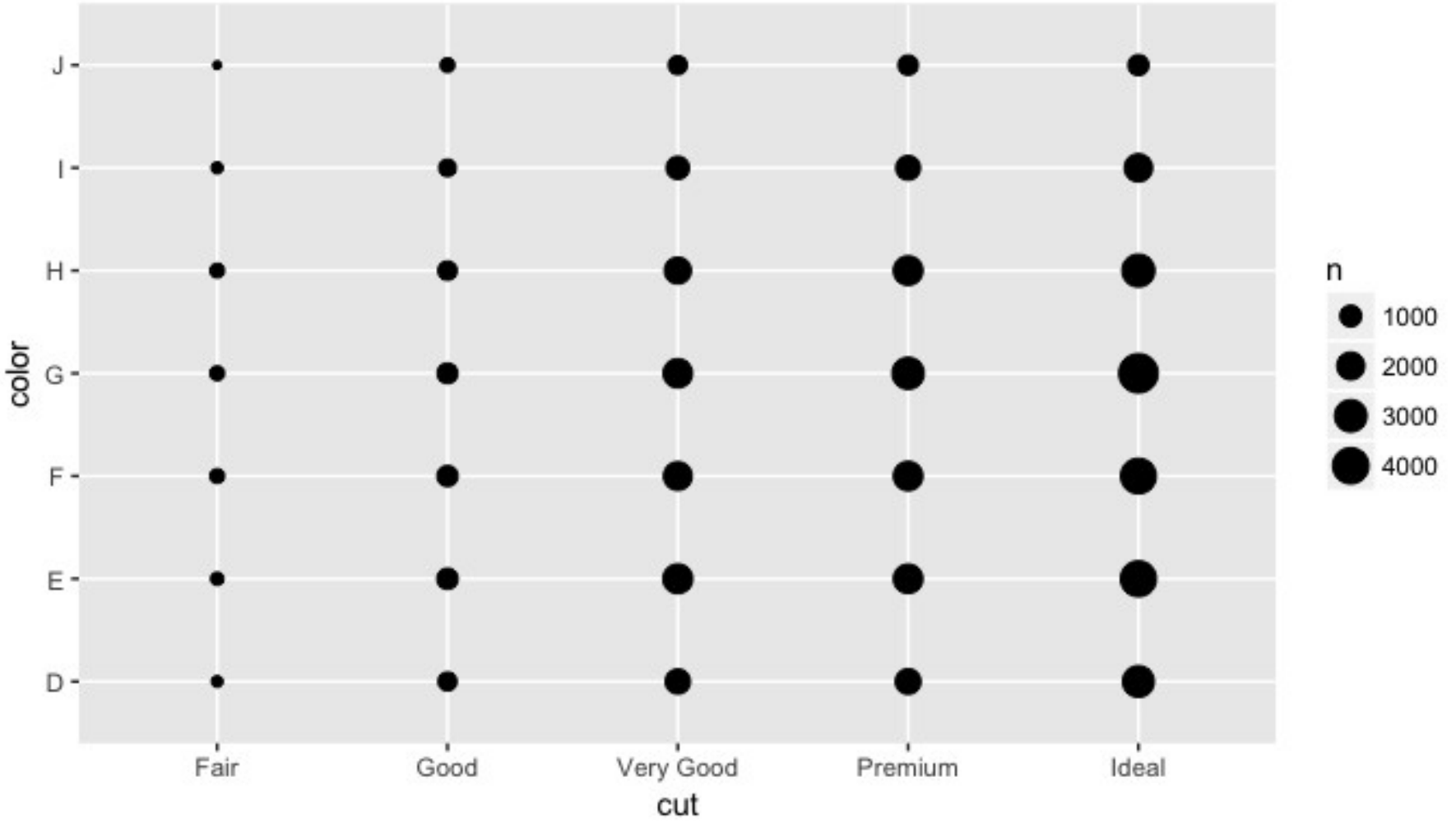
- Pro
  - Box plots are more compact for convenient comparison
- Cons
  - Much less information about the *cut* distribution
  - Be careful, we could incorrectly conclude that better quality diamonds are cheaper on average!

Two Categorical Variables



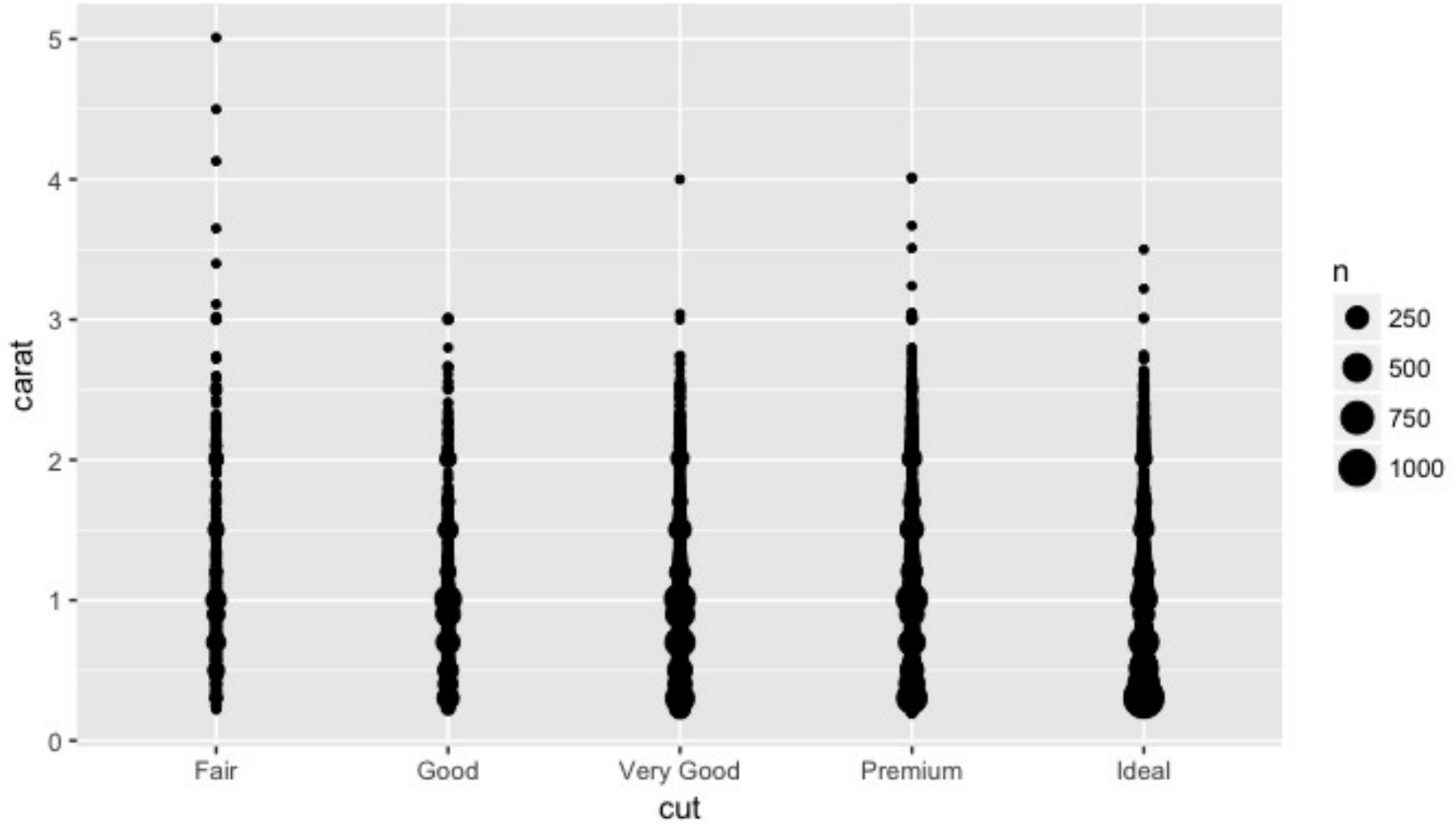

```
# Visualize the covariation between categorical variables with a “Plot of Dots” to determine observations.

ggplot(data = diamonds) +
geom_count(mapping = aes(x = cut, y = color))

# Note:  The size of each circle in the plot displays how many observations occurred at each combination of values

# Get exact text details of the plot

diamonds %>% count(color, cut)
```

Mini Distributions: Cut vs Color



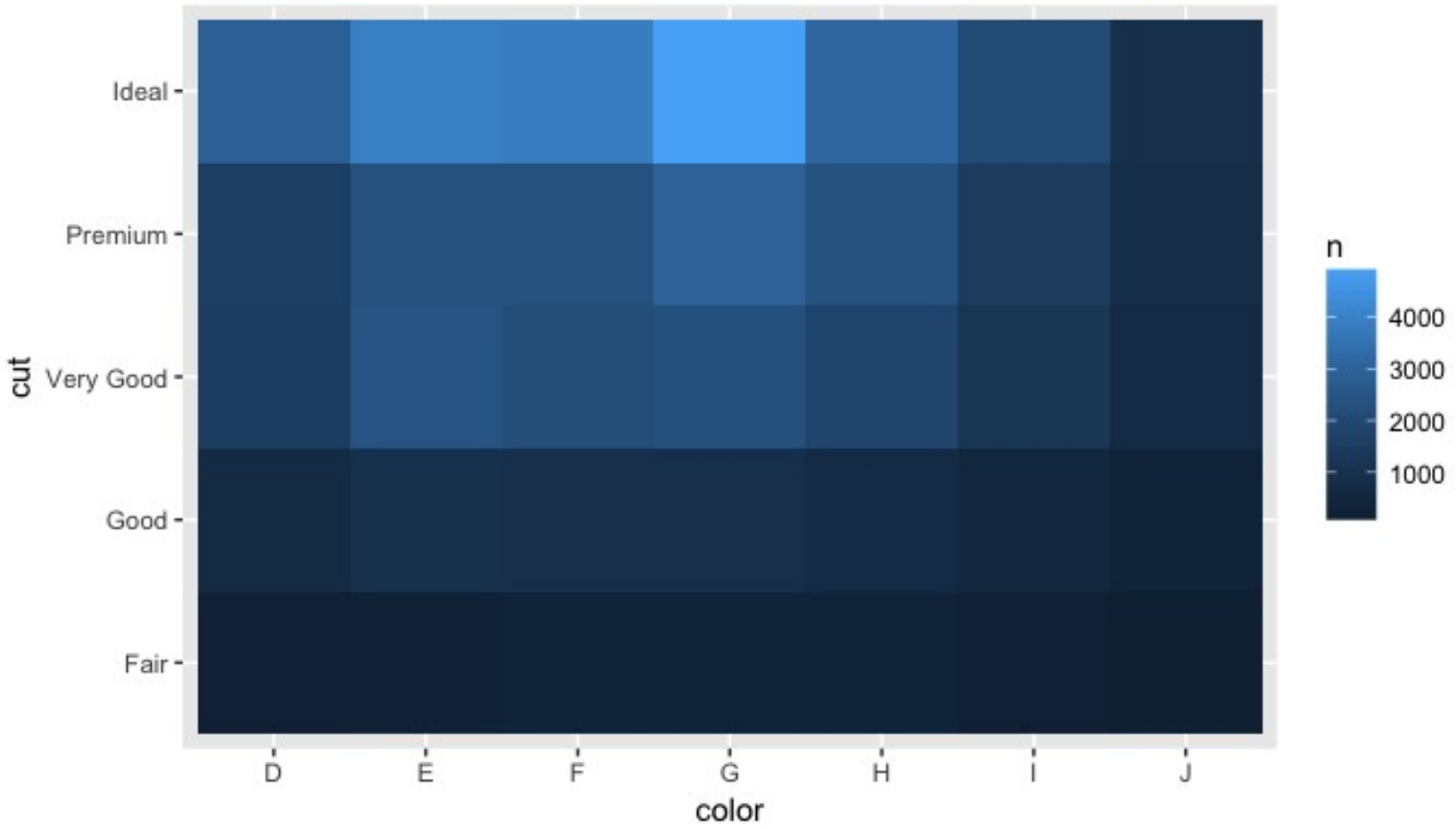

```
ggplot(data = diamonds) +
    geom_count(mapping = aes(x = cut, y = color))
```

Mini Distributions: Cut vs Carat



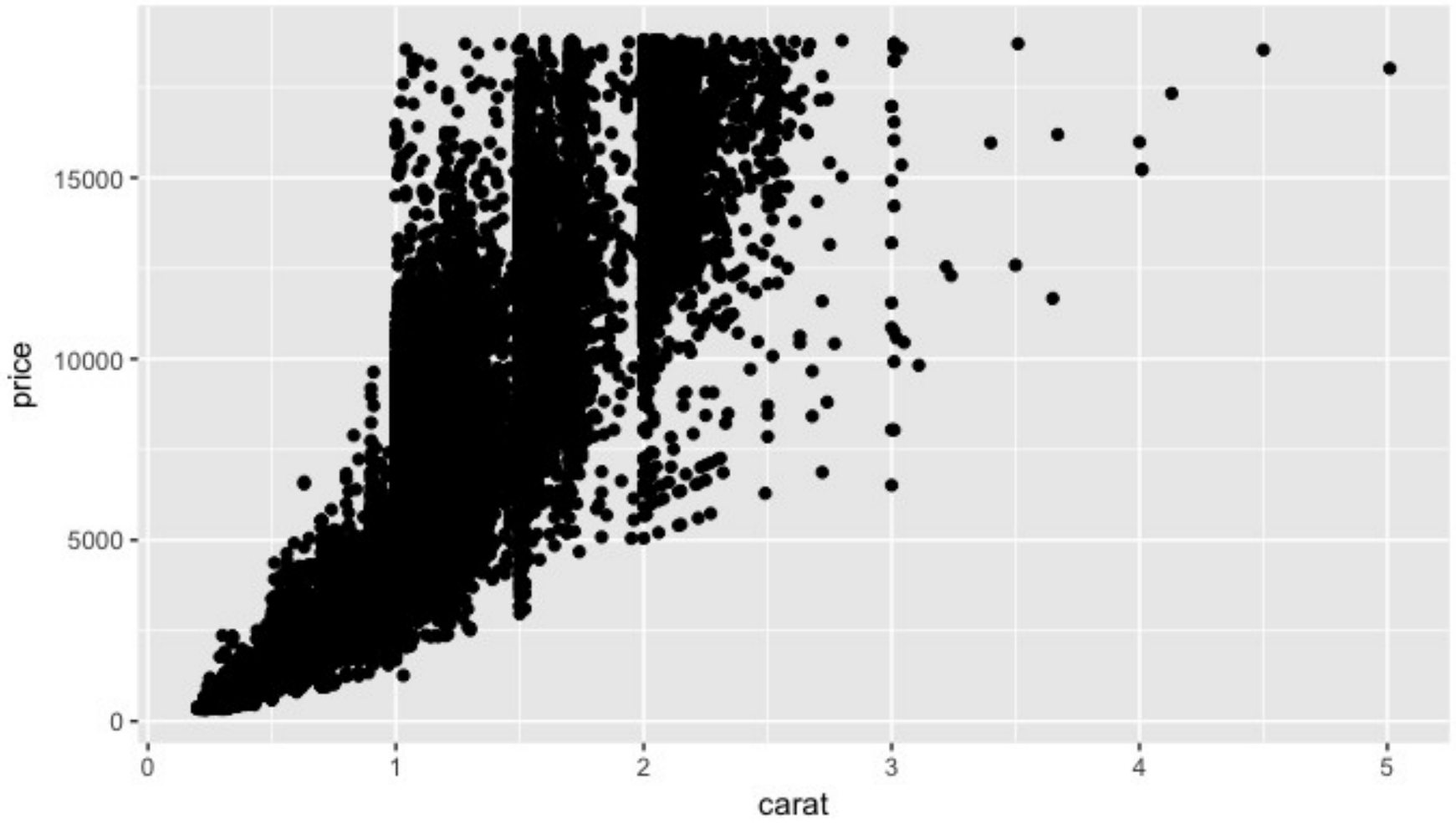

```
ggplot(data = diamonds) +
   geom_count(mapping = aes(x = cut, y = carat))
```

Mini Distributions: Cut vs Color



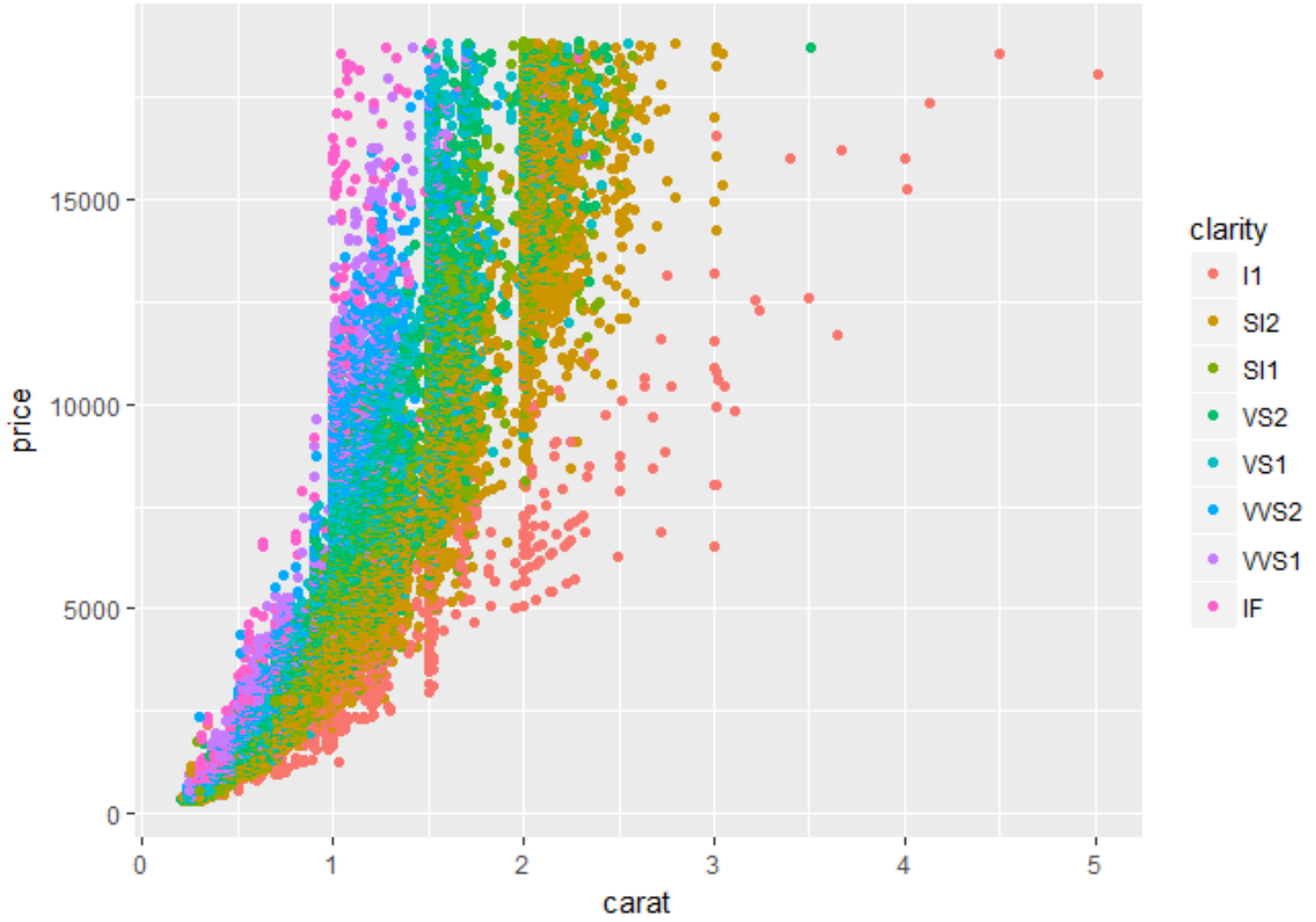

```
diamonds %>%
  count(color, cut) %>%
  ggplot(mapping = aes(x = color, y = cut)) +
    geom_tile(mapping = aes(fill = n))
```

Mini Distributions: Carat vs Price



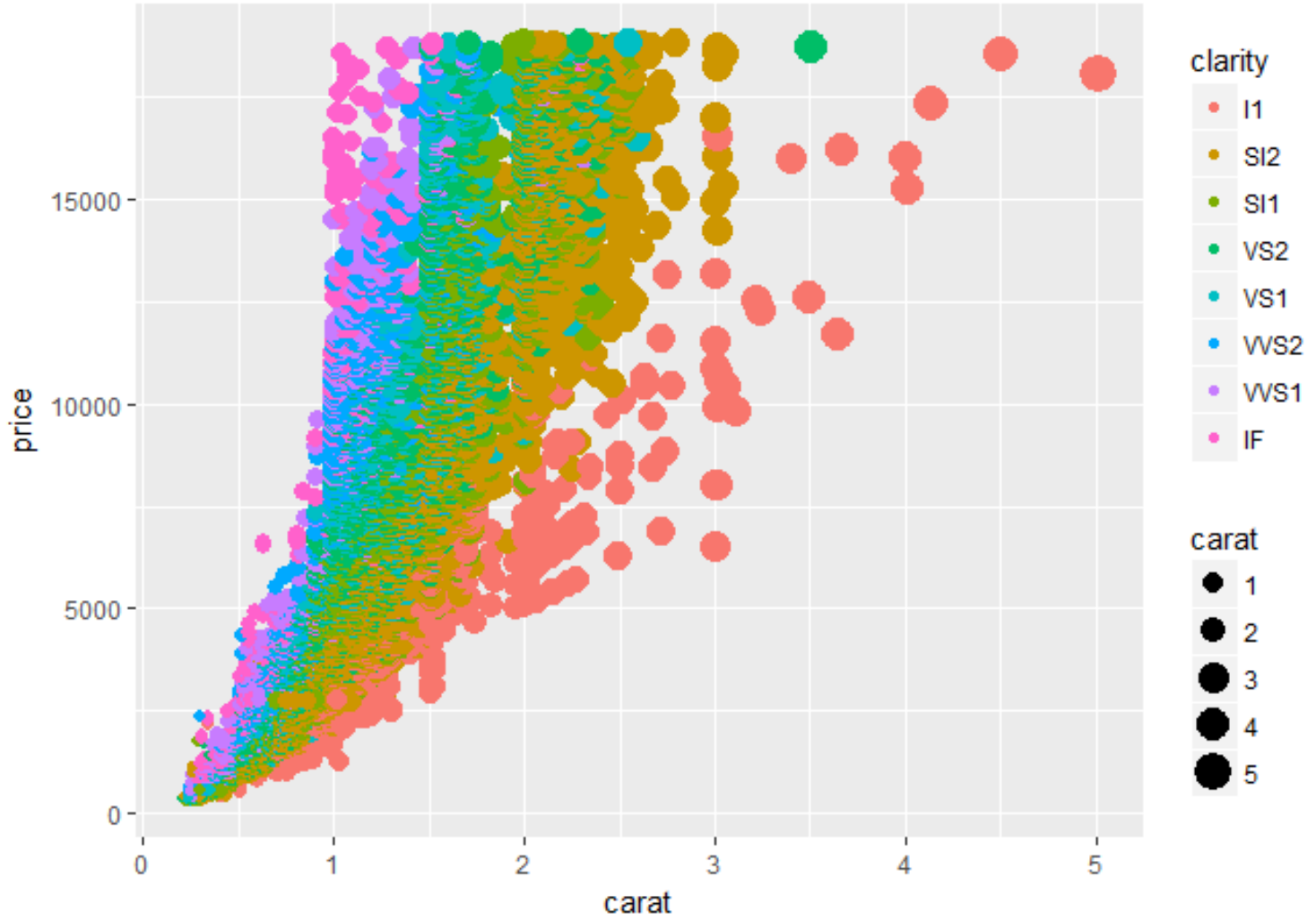

```
ggplot(data = diamonds) +
  geom_point(mapping = aes(x = carat, y = price))
```

Mini Distributions: Carat vs Price



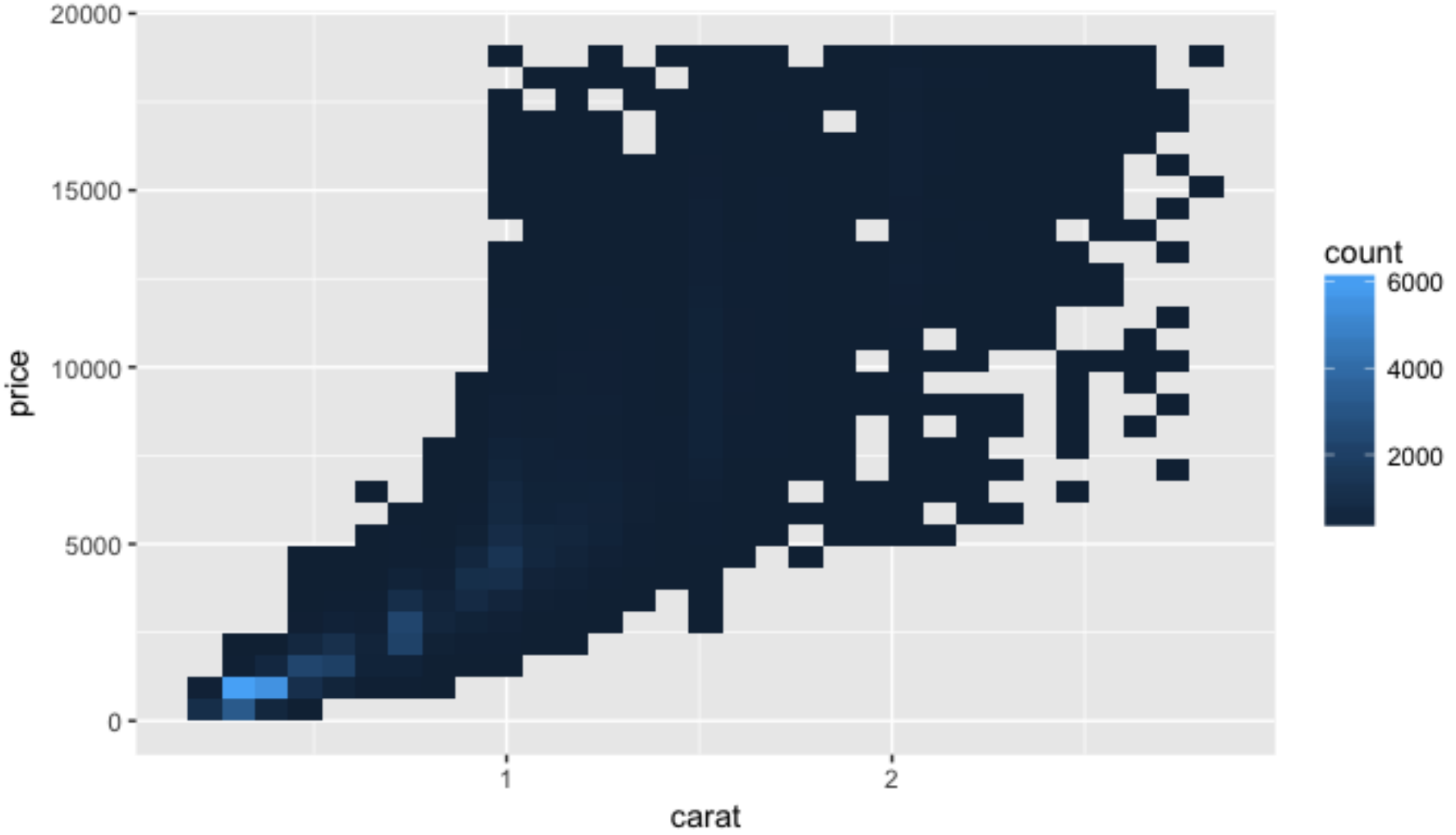

```
ggplot(data = diamonds) +  geom_point(mapping = aes(x = carat, y = price, color= clarity))
```

Mini Distributions: Carat vs Price




```
ggplot(data = diamonds) +  geom_point(mapping =
aes(x = carat, y = price,color= clarity, size = carat))
```

Mini Distributions: Carat vs Price




```
ggplot(data = smaller) +
  geom_bin2d(mapping = aes(x = carat, y = price))
```

Consider This: Subset *plots*

- Can you plot your diamond dataset with different subsets of data? Compare your plots!
- Play with the below code to see what you can isolate to plot

```
diamonds3 <- diamonds %>%
  mutate(y = ifelse(y < ## | y > ##, NA, y))
```

- Time permitting, can you find another dataset apply to plots?